

SYLLABUS

(PSU + M.Tech + ISRO/DRDO)

UNIT 1: Fundamentals of Cloud Computing

1.1 Evolution of Computing

- Mainframe Computing
- Client-Server Computing
- Distributed Computing
- Cluster Computing
- Grid Computing
- Utility Computing
- Transition to Cloud Computing

1.2 Cloud Basics

- Definition (NIST definition)
- Characteristics:
 - On-demand self-service
 - Broad network access
 - Resource pooling
 - Rapid elasticity and scalability
 - Measured service (pay-as-you-go)

1.3 Core Concepts

- Multi-tenancy
- Scalability vs Elasticity
- Pay-as-you-go model
- Utility computing concept

1.4 Advantages and Limitations

- Cost efficiency
- Flexibility and scalability
- Reliability
- Security and privacy concerns
- Vendor lock-in

1.5 Comparison Topics (Very Important)

- Cloud vs Traditional Computing
 - Cloud vs Distributed Computing
 - Grid vs Cloud Computing
-

UNIT 2: Service Models and Deployment Models

2.1 Service Models (SPI Model)

- Infrastructure as a Service (IaaS)
- Platform as a Service (PaaS)
- Software as a Service (SaaS)

2.2 Emerging Service Models

- Function as a Service (FaaS) / Serverless
- Database as a Service (DBaaS)

2.3 Deployment Models

- Public Cloud
- Private Cloud
- Hybrid Cloud
- Community Cloud

2.4 Key Comparisons

- IaaS vs PaaS vs SaaS
 - Public vs Private vs Hybrid
-

UNIT 3: Virtualization (Core Pillar – Highest Priority)

3.1 Fundamentals

- Concept and need of virtualization
- Hardware vs Software virtualization

3.2 Hypervisors (VMM)

- Type 1 (Bare-metal)
- Type 2 (Hosted)

3.3 Virtualization Techniques

- Full Virtualization
- Para-virtualization

3.4 Types of Virtualization

- CPU/Compute Virtualization
- Memory Virtualization
- Storage Virtualization:
 - SAN, NAS, DAS
- Network Virtualization

3.5 VM Management

- Virtual Machine lifecycle
- VM provisioning
- VM placement
- VM live migration

3.6 Containerization

- Virtual Machines vs Containers
 - Docker basics
 - Container orchestration (Kubernetes fundamentals)
-

UNIT 4: Cloud Architecture and Infrastructure

4.1 Data Center Architecture

- Components of data centers
- Design considerations

4.2 Distributed Architecture

- Distributed systems basics
- Resource pooling

4.3 Reliability and Performance

- Fault tolerance
- High availability
- Disaster recovery

4.4 Load Balancing

- Need for load balancing
- Types:
 - Hardware vs Software
- Algorithms:
 - Round Robin
 - Least Connections

4.5 Architectural Models

- N-tier architecture

4.6 Agreements and Metrics

- Service Level Agreements (SLA)
 - Performance metrics
-

UNIT 5: Cloud Storage and Data Management

5.1 Storage Models

- Block Storage
- File Storage
- Object Storage

5.2 Distributed File Systems

- HDFS (Hadoop Distributed File System)
 - NameNode
 - DataNode
 - Replication
- Google File System (GFS)

5.3 Database Systems

- RDBMS in cloud
- NoSQL databases:
 - Key-value stores
 - Column-oriented
 - Document-based

5.4 Data Consistency and Models

- CAP Theorem
- BASE properties
- Consistency models

5.5 Big Data Frameworks

- MapReduce paradigm

- Hadoop ecosystem basics
-

UNIT 6: Networking in Cloud

6.1 Cloud Networking Basics

- Virtual Private Cloud (VPC)
- Subnets and routing

6.2 Network Components

- DNS
- Load balancers
- Content Delivery Networks (CDN)

6.3 Advanced Networking

- Software Defined Networking (SDN)
 - Network Function Virtualization (NFV)
 - Network virtualization
-

UNIT 7: Cloud Resource Management and Scheduling

7.1 Resource Provisioning

- Static provisioning
- Dynamic provisioning
- Over-provisioning vs Under-provisioning

7.2 Resource Allocation

- Scheduling techniques
- Resource allocation policies

7.3 VM Resource Management

- VM placement strategies
 - VM migration techniques
-

UNIT 8: Cloud Security and Trust Management (Very Important)

8.1 Security Fundamentals

- CIA Triad:
 - Confidentiality
 - Integrity
 - Availability

8.2 Identity and Access Management

- Authentication vs Authorization
- IAM
- Single Sign-On (SSO)
- Role-Based Access Control (RBAC)

8.3 Data Security

- Encryption:
 - At rest
 - In transit
- Key management

8.4 Threats and Vulnerabilities

- Data breaches
- DDoS attacks
- Insider threats
- Multi-tenancy risks

8.5 Security Models

- Shared responsibility model

- Security in IaaS vs SaaS

8.6 Compliance and Trust

- SLAs and security policies
 - Auditing
 - Trust models
-

UNIT 9: Cloud Programming and Technologies

9.1 Distributed Computing Concepts

- Basics of distributed computing

9.2 Web Services

- Service-Oriented Architecture (SOA)
- REST APIs
- SOAP

9.3 Application Architecture

- Microservices architecture
- Stateless systems

9.4 APIs

- REST principles
 - HTTP methods
-

UNIT 10: DevOps and Cloud Integration

10.1 DevOps Basics

- CI/CD pipeline

- Continuous integration
- Continuous deployment

10.2 Infrastructure as Code

- IaC concepts

10.3 Monitoring and Logging

- Logging systems
- Monitoring tools

10.4 Tools (Conceptual)

- Jenkins
 - Kubernetes
-

UNIT 11: Advanced and Emerging Topics

11.1 Serverless Computing

- FaaS (Function as a Service)

11.2 Edge and Fog Computing

- Architecture
- Use cases (IoT, low latency systems)

11.3 Cloud Simulators

- CloudSim basics

11.4 Distributed System Algorithms (M.Tech Focus)

- Consensus algorithms:
 - Paxos
 - Raft

UNIT 12: Cloud Economics and SLA

12.1 Cost Models

- Pay-as-you-go
- Subscription models

12.2 Billing

- Pricing strategies

12.3 SLA Parameters

- Availability (e.g., 99.9%, 99.99%)
 - Throughput
 - Latency
-

UNIT 13: Cloud Platforms (Conceptual Overview)

13.1 Major Providers

- AWS:
 - EC2
 - S3
 - IAM
- Microsoft Azure
- Google Cloud Platform

13.2 Services Overview

- Compute services
- Storage services
- Identity services
- Serverless services

HIGH-PRIORITY TOPICS (FOR PSU / ISRO / DRDO)

Focus most on:

- Virtualization (Hypervisors, VM vs Containers)
 - Service and Deployment Models
 - Cloud vs Distributed/Grid systems
 - Cloud Security (IAM, threats, encryption)
 - Storage (HDFS, NoSQL, CAP theorem)
 - Load balancing and resource management
 - SLA and cloud economics
 - REST APIs and Web services
 - SDN and Network virtualization
-

UNIT 1: Fundamentals of Cloud Computing

1.1 Evolution of Computing

The evolution of computing has transitioned from centralized Mainframe Computing (1960s) toward decentralized models, including Distributed and Cluster Computing (1970s-80s) for better scalability. This progressed into Grid and Utility Computing (1990s) for resource sharing, eventually culminating in modern Cloud Computing, which delivers on-demand, scalable, and pay-per-use services over the internet.  GeeksforGeeks +6

- **Mainframe Computing (1960s-1970s):** Centralized, highly powerful computers serving many users via terminals. It was expensive but efficient for large-scale data processing, as detailed in [this PDF by the الجامعة المستنصرية](#).
- **Client-Server Computing (1980s):** Shifted away from centralized control. Clients (desktops) request resources, and servers provide them, improving user accessibility.
- **Distributed Computing (1970s-80s):** Multiple independent computers, potentially in different locations, working together to achieve a common goal, increasing reliability and performance.
- **Cluster Computing (1980s-90s):** Linked, similar computers (nodes) connected via high-speed networks, acting as a single system to provide a cheaper alternative to mainframes for high-performance computing, according to [GeeksforGeeks](#).
- **Grid Computing (1990s):** Connects geographically dispersed, heterogeneous resources (nodes) over the internet to solve complex tasks. Often called "On-Demand Computing".
- **Utility Computing (2000s):** A model where computing services (storage, processing) are offered on demand and charged based on usage, similar to utility services like electricity, notes [Scribd](#).
- **Transition to Cloud Computing (2000s-Present):** The ultimate culmination of these technologies (virtualization, grids, utility), providing scalable, elastic, and on-demand IT services, as explained in [this preprints.org article](#).  GeeksforGeeks +11

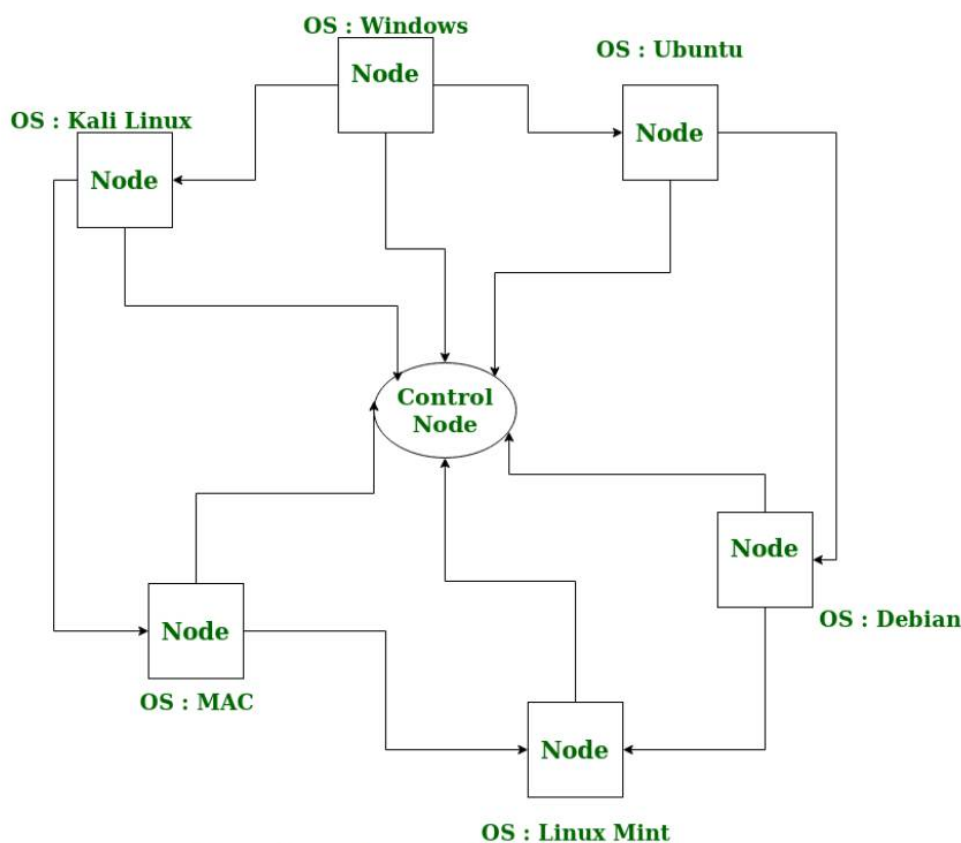
- **Grid computing**

<https://www.youtube.com/watch?v=aJil9-1FnuU>

Grid Computing can be defined as a network of computers working together to perform a task that would rather be difficult for a single machine. All machines on that network work under the same protocol to act as a virtual supercomputer. The tasks that they work on may include analyzing huge datasets or simulating situations that require high computing power.

Computers on the network contribute resources like processing power and storage capacity to the network.

Grid Computing is a subset of distributed computing, where a virtual supercomputer comprises machines on a network connected by some bus, mostly [Ethernet](#) or sometimes the Internet. It can also be seen as a form of [Parallel Computing](#) where instead of many CPU cores on a single machine, it contains multiple cores spread across various locations. The concept of grid computing isn't new, but it is not yet perfected as there are no standard rules and protocols established and accepted by people.



Topology in Grid Computing

• Distributed vs Grid computing

Distributed Computing

A system where multiple computers **work together as a single system** to solve a problem.

- Tightly or loosely connected
- Usually under **one organization/control**

Grid Computing

A type of distributed computing where **geographically distributed, independent systems** collaborate to solve **large-scale problems**.

- Loosely coupled
- Across **multiple organizations**

- **Distributed Computing**
→ "Many machines working together like one system"
- **Grid Computing**
→ "Many independent systems sharing resources across organizations"

3. Detailed Comparison

Feature	Distributed Computing	Grid Computing
Definition	Multiple systems working as one	Collection of independent systems collaborating
Control	Single organization	Multiple organizations
Coupling	Can be tightly coupled	Loosely coupled
Location	Usually same location or controlled network	Geographically distributed
Resource Sharing	Controlled and coordinated	Open and dynamic
Heterogeneity	Moderate	High (different systems, OS, networks)
Goal	General-purpose computing	Large-scale scientific/computational problems
Management	Centralized or semi-centralized	Decentralized
Network	High-speed network (LAN)	Internet-based / WAN
Example Use	Web servers, distributed databases	Scientific simulations, research computing

4. Architecture Difference

Distributed Computing

- Nodes are:
 - Closely connected
 - Coordinated centrally

Flow:

System → Multiple nodes → Unified output

Grid Computing

- Nodes are:
 - Independent
 - Join/leave dynamically

Flow:

Different organizations → Share resources → Solve big task

5. Key Technical Differences

1. Resource Ownership

- Distributed → Same owner
 - Grid → Different owners
-

2. Coordination

- Distributed → Pre-planned coordination
 - Grid → Dynamic coordination
-

3. Scheduling

- Distributed → Central scheduler
 - Grid → Distributed schedulers
-

4. Fault Handling

- Distributed → Controlled environment
- Grid → Unpredictable (nodes may leave anytime)

Example: Distributed Web Application (like a large e-commerce backend)

How it works

- System is split into multiple services:
 - User service
 - Payment service
 - Order service
- Each runs on different machines
- All are controlled by **one organization**

Flow

1. User places order
2. Request goes to **API server**
3. API calls:
 - Payment service
 - Inventory service
4. All services coordinate and return result

Key Technical Point

- Systems communicate via:
 - APIs
 - RPC calls
- Work together as **one system**

Why this is Distributed (not Grid)

- Single organization
- Centrally managed
- Tight coordination



Example: Scientific Research Computation (like protein folding or space data analysis)

How it works

- A huge task is divided into small parts
- Sent to computers across:
 - Universities
 - Research labs
 - Different countries

Flow

1. Big task → split into chunks
2. Each organization's system processes a chunk
3. Results are sent back
4. Combined into final output

Key Technical Point

- Uses **middleware/grid scheduler**
- Nodes are:
 - Independent
 - Heterogeneous

Why this is Grid

- Multiple organizations
- Loosely connected
- Dynamic participation

• Grid vs Cloud computing

Grid Computing

Resource sharing across multiple independent systems to solve large problems

Cloud Computing

On-demand delivery of computing resources (compute, storage, network) over the internet

C. Detailed Comparison

Feature	Grid Computing	Cloud Computing
Ownership	Multiple organizations	Single provider (centralized)
Goal	Solve large scientific problems	Provide scalable services
Resource Allocation	Decentralized	Centralized
Availability	Depends on participants	Highly reliable
Scalability	Limited, less flexible	Highly scalable (elastic)
Virtualization	Not essential	Core concept
Pricing	Usually free/research-based	Pay-as-you-go
Management	Distributed	Managed by provider
Access	Collaborative	Service-based (on-demand)

D. Architecture Difference

Grid Computing

- No central control
- Uses **middleware**
- Nodes join/leave dynamically

Cloud Computing

- Centralized data centers
- Uses:
 - Virtualization
 - Containers
- Managed infrastructure

Grid Example

- A climate simulation job:
 - Runs on 1000 computers across universities
 - Each runs part of simulation

Cloud Example

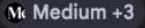
- A company deploys web app:
 - Uses virtual machines
 - Scales automatically when traffic increases

4. One-Line Differences (Very Important)

- **Grid vs Distributed**
→ Grid = cross-organization distributed system
- **Grid vs Cloud**
→ Grid = resource sharing
→ Cloud = service provisioning

1.2 Cloud Basics

According to the [NIST Definition \(SP 800-145\)](#), cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications) that can be rapidly provisioned and released with minimal management effort or service provider interaction.  NIST Computer Security Resource Center ...

The National Institute of Standards and Technology (NIST) defines five essential characteristics for cloud computing, which are: 

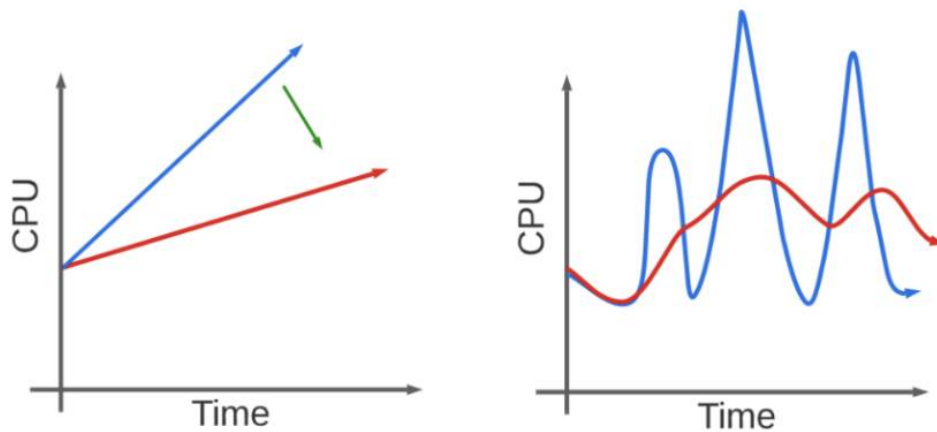
- **On-demand self-service:** Users autonomously procure computing capabilities like server time or network storage without human interaction with the provider.
- **Broad network access:** Services are accessible via standard network mechanisms across diverse devices such as mobile phones, laptops, and workstations.
- **Resource pooling:** The provider uses a multi-tenant model to aggregate computing resources, dynamically assigning them to consumers based on demand.
- **Rapid elasticity and scalability:** Resources can be quickly and often automatically scaled up or down to match demand.
- **Measured service:** Cloud systems leverage metering to monitor, control, and report resource usage, ensuring transparency and enabling pay-as-you-go billing.

 DEV Community +5

1.3 Core Concepts

- **Multi-tenancy:** Multiple users ("tenants") share the same physical infrastructure, yet their data remains separate and secure, allowing for cost-effective, scalable services.
- **Scalability vs. Elasticity:**
 - **Scalability** is the ability of a system to handle increased load by adding resources (e.g., upgrading a server) over a long term.
 - **Elasticity** is the ability to automatically add or remove resources in real-time, matching sudden fluctuations in workload.
 - *Analogy:* Scaling is upgrading a truck to handle more cargo; Elasticity is adding cars to a train during rush hour and removing them afterward.
- **Pay-as-you-go Model:** A pricing model (part of Utility Computing) where users pay only for the resources they actually use—such as CPU time or storage space—rather than paying for upfront hardware.
- **Utility Computing Concept:** Treats computing services (servers, storage, apps) as a utility, like electricity or water. Resources are provided on demand, reducing the need for firms to maintain their own physical, expensive hardware. 

- **Elasticity vs Scalability**



SCALABILITY vs ELASTICITY

Scalability

Scalability is the ability of a system, network, or process to handle growing amounts of work, or its potential to be enlarged in order to accommodate that growth. It refers to the capacity of a system to increase its performance or efficiency as the demand for its services grows over time.

Two things I want to point out here

- increase its performance or efficiency as demand increases
- over time

Over time (months-years) there is an expectation from any system that it will grow, As the system grows, the number of users increases, the feature requirements increase, and thus the resource consumption increases as well

Elasticity

Elasticity consists of the ability of a system to tackle changes in the workload, these changes are outliers considering the systems' average workload and typically occur for a short period.

Keys terms here are

1. tackle changes in the workload
2. occur for a short period

Let's take an example of a normal system, its resource usage will vary if observed over a short period (hours or days). with spikes occurring at certain times. It is easy to understand why this happens. Take an e-commerce site for example, when there is a Black Friday sale or Christmas coming up, the system used will be much higher than usual. Or a ticketing site, when a top singer is going to have a concert, will also attract a large number of people to come to grab tickets

	Cloud Elasticity	Cloud Scalability
1	Elasticity is used just to meet the sudden up and down in the workload for a small period of time.	Scalability is used to meet the static increase in the workload.
2	Elasticity is used to meet dynamic changes, where the resources need can increase or decrease.	Scalability is always used to address the increase in workload in an organization.
3	Elasticity is commonly used by small companies whose workload and demand increases only for a specific period of time.	Scalability is used by giant companies whose customer circle persistently grows in order to do the operations efficiently.
4	It is a short term planning and adopted just to deal with an unexpected increase in demand or seasonal demands.	Scalability is a long term planning and adopted just to deal with an expected increase in demand.

1.4 Advantages and Limitations

Advantages of Cloud Computing

- **Cost Efficiency (Cost-Effectiveness):** Eliminates upfront capital expenditure (CapEx) for hardware and software, shifting to operational expenditure (OpEx) with a "pay-as-you-go" model.
- **Flexibility and Scalability:** Resources can be scaled up or down instantly based on demand, providing agility for fluctuating workloads.
- **Reliability & Performance:** High availability, automated backups, and disaster recovery options enhance reliability compared to on-premises systems.
- **Security & Privacy:** Providers invest in top-tier security, including encryption, automated monitoring, and regular updates, often providing better security than in-house IT.
- **Accessibility:** Allows instant access to data and applications from anywhere with an internet connection, fostering collaboration.  Spanning +5

Limitations of Cloud Computing

- **Vendor Lock-In:** Relying on proprietary technologies makes switching cloud providers complex and costly, hindering flexibility.
- **Security & Privacy Concerns:** Storing sensitive data on third-party servers raises data privacy, unauthorized access, and compliance issues.
- **Internet Dependency:** Stable, high-speed internet is required; outages or poor connectivity can halt operations.
- **Unforeseen Costs:** While generally cost-effective, unexpected expenses can arise from data transfers, over-provisioning, or lack of proper monitoring.
- **Limited Control:** The underlying infrastructure is managed by the provider, resulting in less control over IT operations.  Google Cloud +4

1.5 Comparison Topics (Very Important)

- Cloud vs Traditional Computing

Feature 	Cloud Computing	Traditional Computing
Resources	Virtualized, shared, on-demand.	Physical hardware (servers).
Scalability	High elasticity (increase/decrease easily).	Limited by physical hardware.
Cost	Pay-as-you-go (OpEx).	High upfront cost (CapEx).
Maintenance	Managed by provider.	User-owned and maintained.
Deployment	Rapid.	Slow/requires installation.

- Cloud vs Distributed Computing

Feature 	Cloud Computing	Distributed Computing
Goal	Deliver computing resources over network.	Achieve a single goal/task together.
Architecture	Centralized management of resources.	Decentralized components.
Structure	Virtualized resources.	Computers connected in a network.
Main Use	On-demand service, storage, webapps.	Complex, massive parallel computations.
Location	Usually data centers.	Geographically dispersed (often).

- Grid vs Cloud Computing

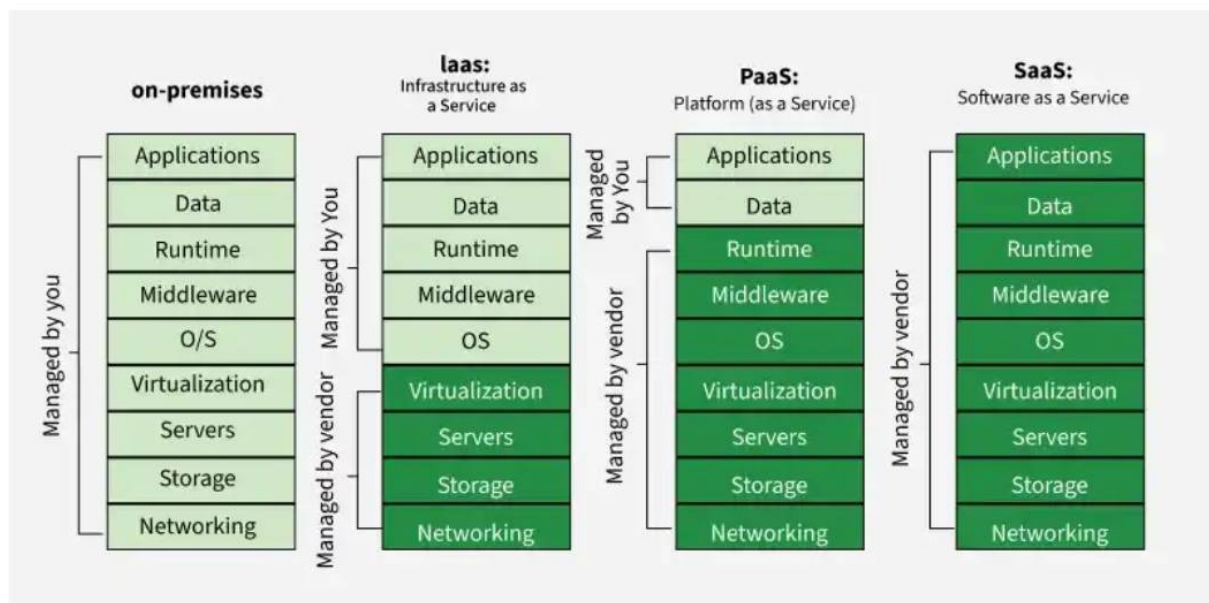
Feature 	Cloud Computing	Grid Computing
Focus	Ease of use, accessibility, services.	High-performance computing (HPC).
Resource Source	Owned by one provider.	Shared from multiple organizations.
Architecture	Centralized/virtualized.	Decentralized.
Flexibility	High (handles different OS/apps).	Low (usually optimized for one task).
Typical Usage	Commercial web apps, data storage.	Scientific research, simulations.

UNIT 2: Service Models and Deployment Models

2.1 Service Models (SPI Model)

Cloud computing provides on-demand IT resources that can easily scale to meet business demands. Rather than maintaining complex physical hardware and software on-premises, organizations can rent these services over the internet.

The cloud ecosystem is divided into three primary service models: IaaS, PaaS, and SaaS. Each model offers a different level of control, flexibility, and management, catering to different business needs from raw infrastructure to ready-to-use software



• Infrastructure as a Service (IaaS)

IaaS provides virtualized computing resources such as servers, storage, and networking over the internet on a pay-as-you-go basis. It offers the highest level of control and flexibility, acting as the digital foundation for your IT operations.

How it works: It is like renting an empty plot of land. The provider gives you the space and the utilities (hardware, network, storage), but you are entirely responsible for building the house (installing the OS, middleware, and applications).

Characteristics:

- Eliminates the capital expense of buying physical servers.
- Offers complete control over operating systems, applications, and development frameworks.
- Resources can be scaled up or down instantly based on traffic spikes.

Real-World Use Case: A tech startup experiencing rapid growth can use IaaS to instantly increase their server capacity to handle web traffic without needing to purchase, house, or wire expensive physical hardware.

Target Audience: IT Administrators, Network Architects, and DevOps teams.

Popular Providers: Amazon Web Services (AWS EC2), Microsoft Azure (Virtual Machines), Google Compute Engine (GCE), DigitalOcean.

• Platform as a Service (PaaS)

PaaS offers a complete, managed cloud environment specifically designed for developing, testing, running, and managing applications. It abstracts the underlying infrastructure so developers can focus purely on writing code.

How it works: It is like renting an unfurnished house. The foundation, plumbing, and electricity are already built and maintained by the landlord; you just need to bring your furniture and decorate (write and deploy your code).

Characteristics:

- Acts as a complete toolkit for developers, providing pre-built tools, libraries, and development environments.
- The cloud provider handles all backend infrastructure, including server provisioning, OS patching, and network routing.
- Automates deployment workflows and allows for easy team collaboration.

Real-World Use Case: A software development team building a mobile app can use PaaS to write their code and automatically connect it to managed databases and authentication services without ever having to configure a backend server.

Target Audience: Software Developers and Application Delivery Teams.

Popular Providers: AWS Elastic Beanstalk, Google App Engine, Heroku, Microsoft Azure App Service.

● **Software as a Service (SaaS)**

SaaS is the most user-friendly and widely used cloud model. It delivers fully functional, ready-to-use software applications over the internet, typically on a subscription basis.

How it works: It is like staying in a fully furnished, serviced hotel room. Everything is provided, maintained, and cleaned for you; you simply walk in and use it.

Characteristics:

- Applications are ready to use immediately directly through a web browser or mobile app.

- The cloud provider handles absolutely everything: infrastructure, software updates, bug fixes, and security maintenance.
- Requires minimal technical expertise to operate.

Real-World Use Case: A global sales team uses a cloud-based CRM to track leads and communicate with clients. They simply log in via their browsers; they don't need to worry about where the data is hosted or how the software is updated.

Target Audience: End-users, Business Professionals, and Consumers.

Popular Providers: Salesforce, Google Workspace (Gmail, Docs), Microsoft 365, Zoom, Slack.



IaaS

Infrastructure as a Service

- Provides servers, storage, networking, virtualization as raw resources
- You manage the OS, Middleware, Runtime, Data & Applications
- Best when you want maximum flexibility & control over infra

Examples:

AWS EC2, Azure VM, Google Compute Engine



PaaS

Platform as a Service

- Provides a ready-to-use platform to build & deploy apps
- The provider manages OS, Middleware, Runtime, Security
- Developers just focus on writing code

Examples:

AWS Elastic Beanstalk
Google App Engine
Azure App Service



SaaS

Software as a Service

- End-users get ready-to-use software directly via browser or app
- No infra, OS, or runtime management required
- Just login and start using

Examples:

Gmail
Salesforce
Microsoft 365, Dropbox

2.2 Emerging Service Models

• Function as a Service (FaaS) / Serverless

FaaS is a core component of serverless computing, where developers run individual code functions in response to specific events without managing servers or infrastructure. The cloud provider automatically handles resource allocation, scaling, and maintenance.  Proactive Data Systems +3

- **Key Characteristics:**

- **Server Management:** The cloud provider manages all physical hardware, virtual machines, and web server software.
- **Event-Driven:** Functions are executed only when triggered by events like HTTP requests, database changes, or file uploads.
- **Stateless:** Functions are typically stateless, meaning they do not store data between invocations, which must be handled by external services.
- **Billing:** Users are billed based on the exact compute time consumed (often in 100-millisecond blocks), rather than for idle capacity.

- **Benefits:**

- **Increased Efficiency:** Developers focus solely on writing code and building application logic, accelerating the development process and reducing time-to-market.
- **Automatic Scaling:** Applications scale automatically and instantaneously based on demand without manual intervention or capacity planning.
- **Cost Efficiency:** The pay-per-use model eliminates costs associated with idle servers and infrastructure management.

Key FaaS Providers & Platforms

- **AWS Lambda:** The most popular provider, used for triggering code via file uploads (S3), database changes (DynamoDB), or API requests.
- **Google Cloud Functions:** Event-driven serverless functions supporting multiple languages like Node.js, Python, and Go.
- **Microsoft Azure Functions:** Known for its flexibility with bindings and triggers, popular for enterprise workflows.
- **IBM Cloud Functions:** A polyglot FaaS platform based on Apache OpenWhisk.
- **Cloudflare Workers:** Enables running code at the edge of the network, closer to users.
- **OpenFaaS:** An open-source framework for deploying serverless functions on Kubernetes.  Red Hat +4

Common FaaS Use Cases

- **Image Processing/Thumbnail Generation:** Triggering a function to resize an image immediately after a user uploads it to cloud storage.
- **IoT Data Processing:** Processing sensor data from IoT devices, cleaning it, and storing it in a database in real-time.
- **Backend Web APIs:** Creating RESTful APIs where code runs only when a request is made, removing the need for a dedicated server.
- **Real-time File Processing:** Validating uploaded files, transcoding videos, or extracting data from uploaded documents.
- **Database Triggers:** Running custom code whenever a new user row is inserted into a database (e.g., sending a welcome email).
- **Scheduled Tasks (Cron Jobs):** Running automated tasks like cleaning up log files or generating daily reports at specific times.  Red Hat +5

Real-World Examples

- **Netflix:** Uses AWS Lambda for building automated media encoding and managing the encoding pipeline.
- **Slack:** Employs FaaS for running chatbot backends and implementing serverless integrations.
- **NASA JPL:** Uses serverless functions to process and analyze telemetry data from spacecraft in real-time.  GeeksforGeeks

• Database as a Service (DBaaS)

DBaaS is a cloud computing service that offers access to database software where the vendor manages the underlying infrastructure, setup, and maintenance. Users interact with the database via a platform, API, or standard tools. 🚀 Aerospike +3

- **Key Characteristics:**

- **Managed Operations:** The service provider handles tasks like provisioning, configuration, security, backups, recovery, and updates.
- **On-Demand Self-Service:** Users can rapidly provision and deprovision database instances through a simple console or API.
- **High Availability:** Data is often replicated across multiple servers or availability zones to ensure continuous access and disaster recovery capabilities.

- **Benefits:**

- **Operational Agility:** Allows for rapid deployment and configuration changes, freeing up IT staff to focus on strategic initiatives.
- **Scalability & Flexibility:** Resources can be scaled up or down easily and quickly based on demand and storage needs.
- **Enhanced Security:** Providers typically offer robust, built-in security features and expert support.

1. Major Cloud Provider Services 🔗

- **Amazon Web Services (AWS):** RDS (relational), DynamoDB (NoSQL), Aurora (high-performance relational), and Redshift (data warehouse).
- **Microsoft Azure:** Azure SQL Database, Azure Cosmos DB (NoSQL), and Azure Database for MySQL/PostgreSQL.
- **Google Cloud Platform (GCP):** Cloud SQL, Cloud Firestore, Cloud Bigtable, and Cloud Spanner.
- **Oracle Cloud:** Oracle Autonomous Database (self-managing and patching). 🌐 TechTarget +4

2. Specialized or Vendor-Managed DBaaS 🔗

- **MongoDB Atlas:** A fully managed NoSQL database service from MongoDB.
- **Redis Cloud:** Managed service for the popular Redis in-memory key-value data store.
- **Snowflake:** A highly popular cloud-native data warehouse as a service.
- **PostgreSQL & MySQL Services:** Various vendors like EDB (Postgres AI) and Crunchy Data (Crunchy Bridge) provide managed SQL solutions. 🌐 IBM +4

2.3 Deployment Models

Cloud deployment models define how infrastructure is owned, managed, and accessed, with four main types: **Public** (shared, third-party), **Private** (exclusive, secure), **Hybrid** (combination), and **Community** (shared by similar entities). Key drivers for selecting a model include cost, compliance, security, and scalability needs.  GeeksforGeeks +2

• Public Cloud

- **Public Cloud:** Infrastructure is owned and managed by third-party providers (e.g., AWS, Azure, Google Cloud) and services are delivered over the internet to the general public.
 - *Pros:* High scalability, cost-effective (no hardware investment), and low maintenance.
 - *Cons:* Less control over data security and compliance.

• Private Cloud

- **Private Cloud:** Infrastructure is used exclusively by a single organization, offering high security, control, and customization. It can be hosted on-premise or managed by a third party.
 - *Pros:* High security, performance, and data privacy.
 - *Cons:* Higher cost and limited scalability compared to public clouds.
- **Examples:** A company-hosted data center for sensitive data or specialized software, such as VMware or OpenStack implementations.

• Hybrid Cloud

1. Definition

Hybrid Cloud = **combination of two or more cloud environments** (typically **private + public cloud**) that remain distinct but **work together**

Key Idea

- Sensitive work → **Private Cloud**
 - Scalable work → **Public Cloud**
-

One-Line Intuition

"Keep critical data private, use public cloud for scalability"

2. Architecture of Hybrid Cloud

Components

- **Private Cloud**
 - On-premise or dedicated infrastructure
- **Public Cloud**
 - Third-party provider (like AWS/Azure)
- **Connectivity**
 - VPN (Virtual Private Network)
 - Dedicated connection

- **Examples:** Using a private cloud for sensitive customer data while using AWS for web application front-end scalability.

Flow Example

1. User request comes
 2. Processed in private cloud
 3. If load increases → shift to public cloud
-

Important Concept: Cloud Bursting

When private cloud is overloaded, workload is shifted to public cloud

3. Key Characteristics

- Integration of multiple environments
 - Data and application portability
 - Scalability (elasticity)
 - Controlled resource usage
-

4. Advantages

- **Flexibility**
 - Use best of both worlds
- **Cost Optimization**
 - Avoid maintaining large private infrastructure
- **Security**
 - Sensitive data stays private
- **Scalability**
 - Public cloud handles peak loads

5. Disadvantages

- Complex management
 - Security challenges in data transfer
 - Integration issues
 - Latency between clouds
-

6. Types of Hybrid Cloud (Important)

1. On-Premise + Public Cloud

- Most common

2. Private Cloud + Public Cloud

- Both cloud-based

3. Multi-Cloud Hybrid

- Multiple public cloud providers

8. Use Cases (Very Important for Exams)

1. Cloud Bursting

- Handle peak traffic

2. Data Backup and Disaster Recovery

- Backup private data to public cloud

3. Development and Testing

- Test in public cloud, deploy in private

4. Big Data Processing

- Store data privately, process in cloud
-

9. Security in Hybrid Cloud

Key Areas

- Data encryption (in transit + at rest)
 - Identity and Access Management (IAM)
 - Secure communication (VPN)
-

Important Concept

Shared Responsibility Model

- Provider → infrastructure security
- User → data/application security

10. Challenges (Exam Focus)

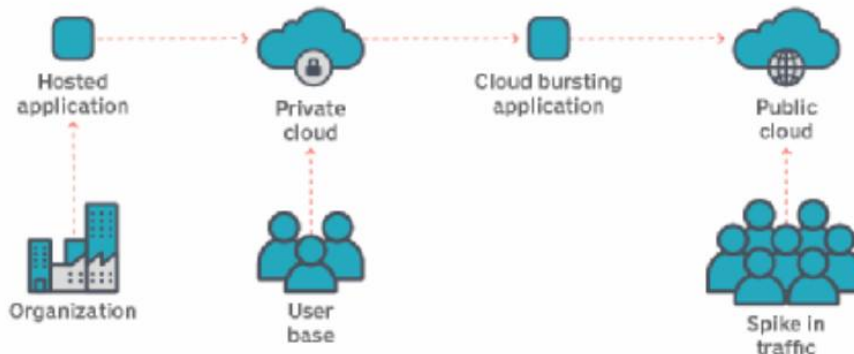
- Data consistency
 - Network latency
 - Interoperability
 - Vendor lock-in
 - Compliance issues
-

11. Hybrid vs Other Deployment Models

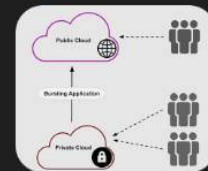
Feature	Private	Public	Hybrid
Control	High	Low	Medium
Cost	High	Low	Optimized
Scalability	Limited	High	High
Security	High	Medium	Balanced

Cloud bursting

Cloud bursting



Cloud bursting is a hybrid cloud configuration where an application runs in a private cloud or on-premises datacenter until capacity limits are reached, at which point it "bursts" into a public cloud (e.g., AWS, Azure) to handle excess demand. This approach allows for scalable, on-demand computing for traffic spikes, ensuring performance without maintaining expensive, permanent infrastructure. [🔗](#)



Key Aspects of Cloud Bursting:

- **Mechanism:** When on-premises capacity hits a set threshold (e.g., 70-80%), additional traffic is automatically redirected to a public cloud, allowing for seamless scaling.
- **Use Cases:** Ideal for seasonal businesses, marketing campaigns, unpredictable web traffic spikes, and intensive tasks like data analytics or rendering.
- **Benefits:** Cost-efficient (pay only for extra capacity used), high availability, and improved application performance during peak demand.
- **Challenges:** Requires careful setup of load balancing and automation tools, potential network latency issues, and security considerations when transferring data to public clouds.
- **Implementation Types:** Automated bursting (using tools to manage resources) and distributed load balancing (sharing traffic between environments). [🔗 Amazon Web Services +8](#)

Key Considerations

While cloud bursting provides flexibility, it is best suited for non-sensitive data and applications designed to be scalable, as transferring sensitive data between environments can raise compliance risks. [🔗 VMware +1](#)

• Community Cloud

- **Community Cloud:** Shared infrastructure among several organizations with common concerns (e.g., security, compliance).
 - *Pros:* Facilitates collaboration, shared costs, and higher security than public clouds.
 - *Cons:* Less scalability than public clouds.  GeeksforGeeks +5

- **Examples:** A shared platform for universities conducting research or a cloud specifically for banks (e.g., SECaaS - Security as a Service).

Feature 	Public Cloud	Private Cloud	Hybrid Cloud	Community Cloud
Ownership	Third-party	Single Organization	Mixed	Shared by Community
Cost	Low (Pay-as-you-go)	High (High CAPEX)	Balanced	Shared Cost
Security	Low-Medium	High	High (Flexible)	Medium-High
Access	Public	Private (Restricted)	Partial	Controlled

UNIT 3: Virtualization

3.1 Fundamentals

• Concept and need of virtualization

Virtualization is the fundamental technology that powers Cloud Computing. It allows you to create multiple simulated environments (Virtual Machines or VMs) from a single physical hardware system.

Before virtualization, a physical server could only run one Operating System (OS) and often one task. This wasted massive amounts of resources if your app only used 10% of the CPU, the other 90% was idle. Virtualization solves this by allowing one physical server to host dozens of virtual servers, each running its own OS and apps isolated from the others.

The Core Architecture

At the heart of virtualization is a piece of software called the **Hypervisor**.

- **Physical Hardware (Host):** The actual server (CPU, RAM, Disk).
- **Hypervisor:** A lightweight software layer that sits between the hardware and the virtual machines. It allocates resources (e.g., "Give VM1 2GB of RAM") and manages the VMs.
- **Virtual Machine (Guest):** A software-based computer that runs like a physical one. It has its own OS, libraries, and applications.

Working of Virtualization

Virtualizations uses special software known as hypervisor, to create many virtual computers (cloud instances) on one physical computer. The Virtual Machines behave like actual computers but use the same physical machine.

Virtual Machines (Cloud Instances)

- After installing virtualization software, you can create one or more virtual machines on your computer.
- Virtual machines (VMs) behave like regular applications on your system.

- The real physical computer is called the **Host**, while the virtual machines are called **Guests**.
- A single host can run multiple guest virtual machines.
- Each guest can have its own operating system, which may be the same or different from the host OS.
- Every virtual machine functions like a standalone computer, with its own settings, programs, and configuration.
- VMs access system resources such as **CPU, RAM, and storage**, but they work as if they are using their own hardware.

Why is Virtualization Important in IT?

- It allows multiple virtual instances to run on a single physical machine, optimizing the use of hardware resources.
- By reducing the need for physical hardware, organizations can save on equipment, maintenance, and energy costs.
- Virtual environments can be easily scaled up or down based on demand.
- It consolidates workloads onto fewer servers, reduces energy consumption, and operational costs.
- It simplifies backup and recovery processes, enhancing data protection and business continuity.

• Hardware vs Software virtualization

Hardware virtualization (using Type 1 hypervisors like ESXi) simulates physical hardware to run multiple full Operating Systems (VMs) with direct hardware access and high performance, ideal for servers. Software virtualization (using Type 2 hypervisors/apps like VirtualBox) runs within a host OS, offering higher flexibility for desktops and testing, but with lower performance.  Fiveable +2

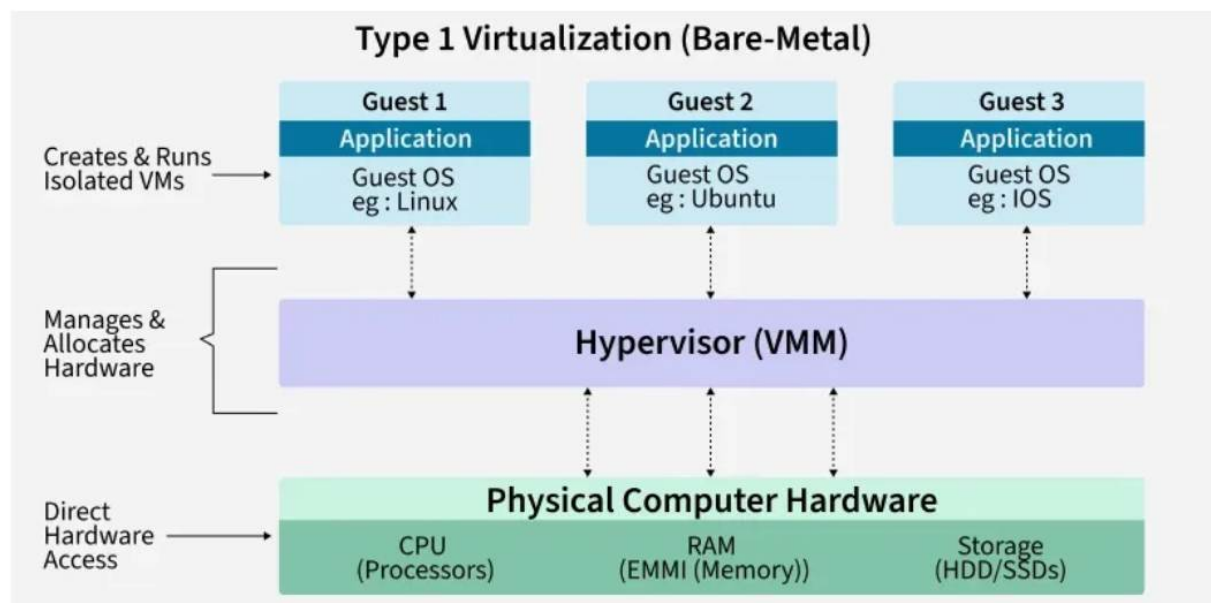
- **Layer of Abstraction:** Hardware virtualization works at the hardware level, creating Virtual Machines (VMs). Software virtualization works at the OS or application level, isolating specific applications.
- **Performance:** Hardware virtualization is more efficient (less overhead) because it often uses hardware-assisted features (Intel VT-x/AMD-V). Software virtualization is generally slower due to the overhead of passing instructions through the host operating system. 
- **Hypervisor Type:** Hardware virtualization typically uses a Type 1 "bare-metal" hypervisor (e.g., [VMware ESXi](#), Microsoft Hyper-V) installed directly on hardware. Software virtualization uses a Type 2 hypervisor (e.g., [Oracle VirtualBox](#), VMware Workstation) running on a host OS.
- **Use Cases:**
 - **Hardware:** Data center consolidation, cloud computing, running multiple OS instances concurrently on one server.
 - **Software:** Software development, testing, running applications from different OSs on a single desktop.
- **Isolation:** Hardware virtualization provides stronger isolation (VMs are completely independent). Software virtualization offers lighter, more flexible isolation, such as containerization.   Fiveable +4

3.2 Hypervisors (VMM - Virtual Machine Monitor)

A Virtual Machine Monitor (VMM), commonly known as a **hypervisor**, is a **software layer that creates, manages, and runs virtual machines (VMs) by virtualizing physical hardware resources**. It allows multiple operating systems to run simultaneously on a single host computer, enabling efficient resource allocation, isolation, and workload consolidation.  ScienceLogic +3

- **Core Responsibilities:** VMMs handle resource partitioning (CPU, memory, storage), isolation between VMs for security, and centralized management of virtualization hosts.
- **Virtualization Benefits:** Enables server consolidation, increased hardware efficiency, and rapid provisioning of virtual environments.  ScienceDirect.com +7

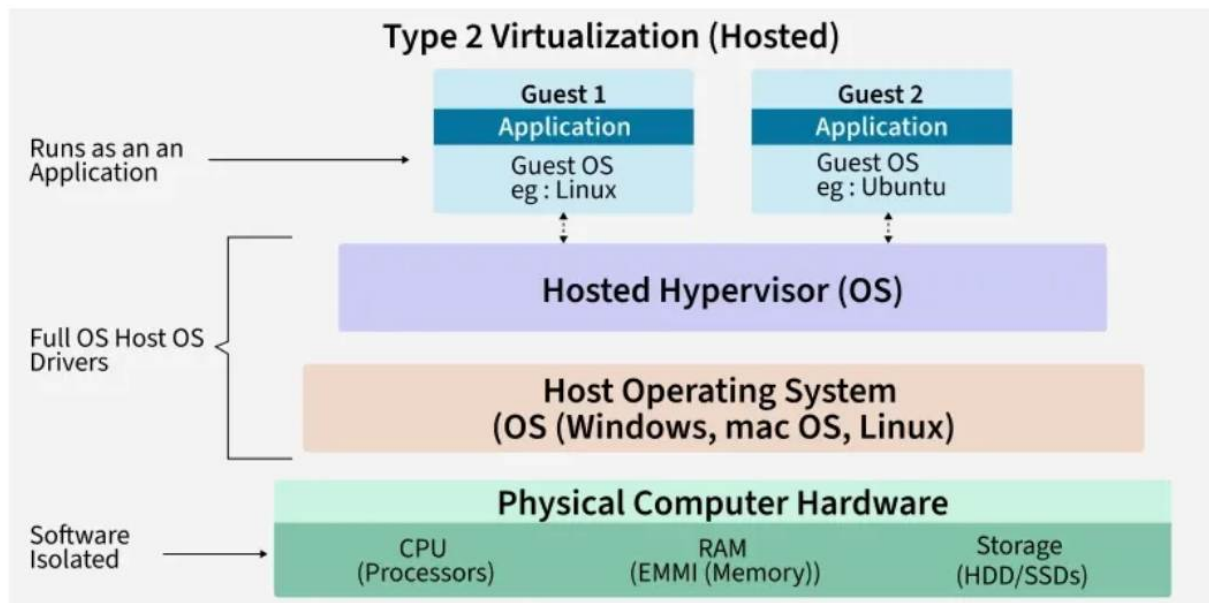
• Type 1 (Bare-metal)



- **How it works:** Installed directly on the physical hardware. There is no host Operating System.
- **Performance:** High. Direct access to hardware resources.

- **Use Case:** Enterprise Data Centers, Cloud Providers (AWS EC2, VMWare ESXi, Microsoft Hyper-V).

- **Type 2 (Hosted)**



- **How it works:** Installed as an application *on top* of an existing OS (like Windows or macOS).
- **Performance:** Lower. Requests must pass through the Host OS first.
- **Use Case:** Personal use, testing labs (Oracle VirtualBox, VMWare Workstation).

3.3 Virtualization Techniques

• Full Virtualization vs Para-virtualization

1. First: What is Virtualization Actually Doing?

A **hypervisor (VMM)** sits between:

- Hardware (CPU, memory, disk)
- Virtual Machines (VMs)

Its job:

- Let multiple OS run on same hardware
- Control access to CPU, memory, I/O

2. The Core Problem

Operating systems (like Linux, Windows) are designed to:

- Run **directly on hardware**
- Execute **privileged instructions** (like changing page tables, managing CPU)

But in virtualization:

- OS is **not in full control**
- Hypervisor must **intercept sensitive instructions**

3. Full Virtualization (No OS Modification Needed)

Idea

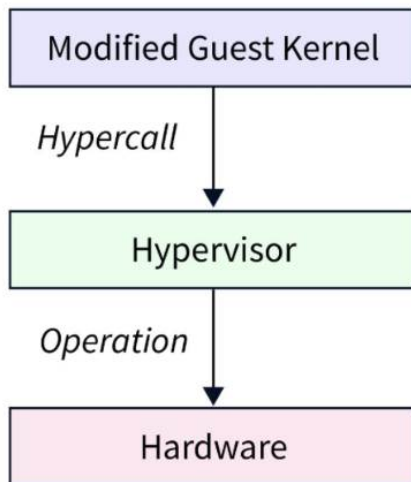
Guest OS thinks:

| "I am running directly on hardware"

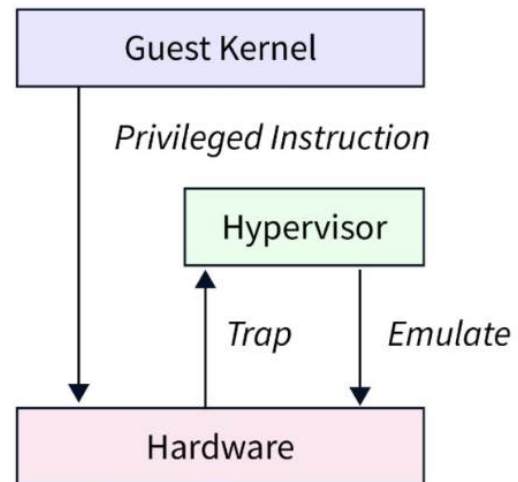
But actually:

| Hypervisor is secretly controlling everything.

Para-virtualization



“Classical” Full Virtualization



How It Works (Step-by-Step)

Step 1: Guest OS runs normally

- Executes instructions (including privileged ones)

Step 2: Sensitive instruction occurs

Example:

- Changing page table
- Accessing hardware

Step 3: TRAP happens

- CPU detects unsafe instruction
- Transfers control to hypervisor
(This is called a **trap**)

Step 4: EMULATE

- Hypervisor **simulates** the instruction safely
(This is called **emulation**)

Trap

- CPU automatically **stops OS**
- Transfers control to hypervisor

Think:

"Stop! Let hypervisor handle this"

Emulate

- Hypervisor **imitates hardware behavior**

Example:

OS says "write to disk"

Hypervisor simulates disk write

Problem

Some CPUs (older x86) don't trap all sensitive instructions.

Solution: Binary Translation

Binary Translation

- Hypervisor **rewrites OS code on the fly**
- Replaces unsafe instructions with safe ones

Example:

Original: privileged instruction
Converted: safe call → hypervisor

Summary of Full Virtualization

- No OS modification
- Uses:
 - Trap + Emulate
 - Binary Translation (if needed)
- Slower (extra overhead)

4. Para-Virtualization (OS is Modified)

Idea

Guest OS knows:

"I am inside a virtual machine"

So instead of doing dangerous things,
it asks the **hypervisor directly for help**

How It Works

Step 1: OS avoids privileged instructions

Instead of:

- Direct hardware access

Step 2: Uses HYPERCALL

Hypercall = API call to hypervisor

Example:

Hypercall: "Hey hypervisor, allocate memory for me"



Instead of:

Direct CPU instruction (unsafe)



No Need For

- Trap
- Binary translation

Because:

- OS cooperates with hypervisor

Summary of Para-Virtualization

- OS is modified
- Uses **hypercalls**
- Faster (less overhead)
- More efficient

6. Full vs Para Virtualization (Comparison)

Feature	Full Virtualization	Para-Virtualization
OS modification	Not required	Required
Awareness	OS unaware	OS aware
Technique	Trap + Emulate, Binary Translation	Hypercalls
Performance	Slower	Faster
Complexity	Higher (hypervisor heavy)	Lower (OS cooperative)

7. One-Line Intuition (Very Important for Exams)

- **Full Virtualization:**
→ "Hypervisor tricks the OS"
- **Para-Virtualization:**
→ "OS cooperates with hypervisor"

8. Real-World Examples (Helpful)

- Full Virtualization:
 - VMware
 - VirtualBox
- Para-Virtualization:
 - Xen (original design)

3.4 Types of Virtualization

1. What "Types of Virtualization" Means

Virtualization = **abstracting (hiding)** physical hardware and creating **virtual versions** of resources.

Types are classified based on **WHAT** is being virtualized:

- CPU
- Memory
- Storage
- Network
- Entire system / OS

• System / Server Virtualization

What it virtualizes

- Entire computer (CPU + memory + OS)

How it works

- Hypervisor creates multiple **Virtual Machines (VMs)**
- Each VM has:
 - Its own OS
 - Its own apps

Flow

Physical Hardware → Hypervisor → Multiple VMs

Key Point

- Each VM is **isolated**

Example

- VMware, VirtualBox

• CPU/Compute Virtualization

What it virtualizes

- CPU (processor time)

How it works

- One physical CPU is shared among multiple VMs
- Hypervisor schedules CPU like an OS scheduler

Technical Idea

- Time slicing:
 - VM1 → VM2 → VM3 → repeat

Important Concept

- vCPU (virtual CPU)

• Memory Virtualization

What it virtualizes

- RAM

How it works

- Each VM thinks it has its own memory
- Hypervisor maps:
 - Virtual memory → Physical memory

Technical Mechanism

- Address translation:
 - Guest Virtual Address → Host Physical Address

Optimization Techniques

- Memory overcommitment
- Ballooning

Memory Overcommitment

- **Concept:** Allocating more memory to virtual machines than the actual physical RAM (RAM) available on the host, allowing for higher consolidation ratios and better resource utilization.
- **Benefits:** Reduces the cost of hardware by allowing more VMs to run per host, ideal for environments with varied memory loads.
- **Risks:** Aggressive overcommitment without sufficient memory reclamation can lead to system instability, excessive swapping, and severe performance degradation. article.nadiapub.com +3

Memory Ballooning

- **Mechanism:** Uses a "balloon driver" (e.g., `vmmemctl`) inside the guest OS that communicates with the hypervisor.
- **Process:**
 1. **Inflation:** When the host needs memory, it signals the driver to inflate, causing it to "balloon" (claim) inactive pages within the guest OS.
 2. **Reclamation:** The guest OS sees high memory usage by the balloon driver and swaps its own idle processes to its own disk, freeing up physical pages.
 3. **Return:** The balloon driver gives these freed physical pages back to the hypervisor, which then reallocates them to other VMs.
 4. **Deflation:** If the guest needs memory back, the balloon deflates, returning memory to the guest OS.
- **Advantages:** This is generally preferred over host-level swapping because it allows the guest operating system to determine which pages are least important, reducing performance penalties.  [arXiv +4](#)

- **Transparent Page Sharing (TPS):** The hypervisor identifies identical memory pages (e.g., common operating system files) across VMs and shares them, reducing the total physical memory needed.

- **Memory Compression:** The hypervisor compresses memory pages before swapping them to disk, which is faster than direct disk I/O.  [GeeksforGeeks +4](#)

- **Shadow Page Tables/Nested Page Tables (NPT/EPT):** Modern CPUs assist the hypervisor in translating guest virtual addresses directly to host physical addresses. This enables nested paging, where the hardware manages mapping guest pages to host frames.

• Storage Virtualization

What it virtualizes

- Storage devices (hard disks)

How it works

- Combines multiple physical disks into **one logical storage pool**

Types

- Block storage (like raw disk)
- File storage
- Object storage

Technologies

- SAN (Storage Area Network)
- NAS (Network Attached Storage)
- DAS (Direct Attached Storage)

Example

- Cloud storage like object storage systems

DAS, NAS, and SAN are the **three main storage technologies**, differing in connectivity, sharing capabilities, and cost. **DAS** is directly connected to a single machine; **NAS** provides shared, file-level storage over a LAN; and **SAN** offers high-speed, block-level shared storage for enterprise environments. [Networkwalks Acade... +2](#)

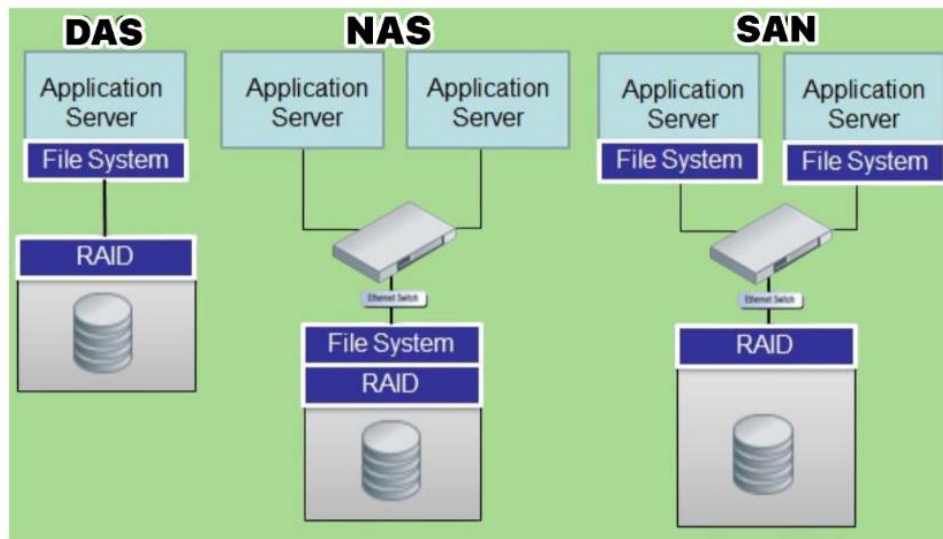
- ✓ **DAS (Direct-Attached Storage)**: Storage is connected directly to a server or workstation via cables (USB, SATA) without a network, offering high performance but limited sharing.
- ✓ **NAS (Network Attached Storage)**: Dedicated file storage devices connected to a network (Ethernet), enabling file-level access for multiple users and devices.
- ✓ **SAN (Storage Area Network)**: A dedicated, high-speed network that connects servers to storage devices, providing block-level access for high-performance applications and enterprise environments. [BMC Software +4](#)

Comparison Table

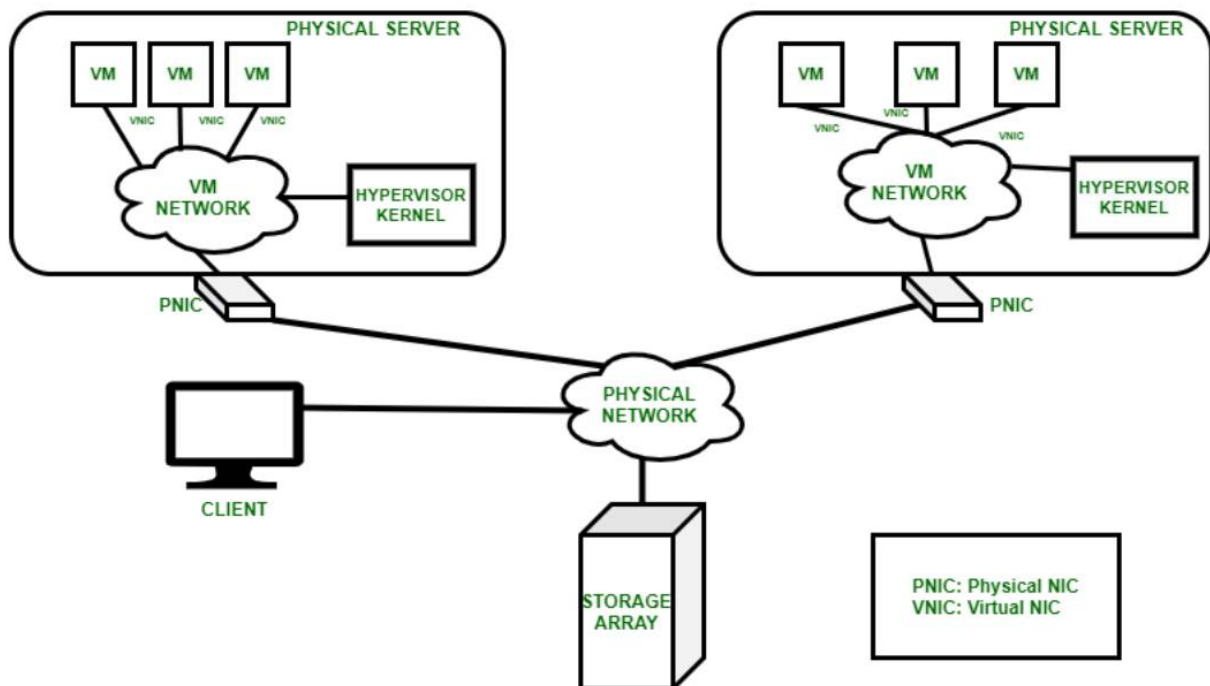
Feature 	DAS	NAS	SAN
Connectivity	<u>Direct (USB, SAS)</u>	<u>LAN (Ethernet)</u>	<u>Dedicated Network (Fiber)</u>
Data Access	<u>Block-level</u>	<u>File-level</u>	<u>Block-level</u>
Sharing	No	Yes (Multiple users)	Yes (Multiple servers)
Cost	Low	Medium	High
Performance	High	Moderate	<u>Very High</u>

Best Use Cases

- **DAS**: Simple storage expansion for a single computer or direct server storage.
- **NAS**: File sharing for small to medium businesses, centralized storage.
- **SAN**: Enterprise applications, databases, high-performance computing. [YouTube +4](#)



• Network Virtualization



What it virtualizes

- Network resources

How it works

- Creates virtual networks on top of physical network

Components

- Virtual switches
- Virtual routers
- Virtual NICs

Advanced Concepts

- SDN (Software Defined Networking)
- NFV (Network Function Virtualization)

Example

- VPC (Virtual Private Cloud)

An overlay network is **a virtual, logical network built on top of an existing physical network** (the underlay). It uses software to create tunnels and abstracted links, allowing for flexible, secure, and independent network management without altering the underlying physical infrastructure. Common examples include **VPNs**, VXLANs, and SD-WAN.  NetBird +4

Network virtualization **decouples network resources from physical hardware, abstracting services like switching, routing, and firewalls into software-based virtual networks**. By creating overlay networks on existing infrastructure, it improves flexibility, security, and scalability, allowing for rapid deployment of virtual machines without manual reconfiguration.  VMware +2

Key Aspects of Network Virtualization:

- **Network Overlay:** Creates virtual layer 2-7 network services that run on top of physical hardware (switches/routers).
- **Types:** Internal (software on a single server, e.g., vSwitches) and External (partitioning physical networks into multiple virtual ones).
- **Technologies:** Involves Software-Defined Networking (SDN) to control the network, and Network Functions Virtualization (NFV) to replace hardware appliances (firewalls, load balancers) with software.  VMware +4

• OS-Level Virtualization (Containers)

What it virtualizes

- Operating System layer

How it works

- Instead of multiple OS:
 - One OS kernel is shared
 - Multiple isolated environments (containers)

Key Difference from VM

- No separate OS per container

Example

- Docker
- Kubernetes (orchestration)

• Desktop Virtualization

What it virtualizes

- User desktop environment

How it works

- Desktop runs on server
- User accesses it remotely

Example

- Remote desktop / VDI

• **Application Virtualization**

What it virtualizes

- Applications

How it works

- App runs in isolated environment
- Doesn't directly depend on OS

Example

- Running apps without installation conflicts

Type	What is Virtualized	Key Idea	
Server/System	Entire machine	Multiple VMs	
CPU	Processor	Time sharing (vCPU)	
Memory	RAM	Address mapping	
Storage	Disks	Storage pooling	
Network	Network resources	Virtual networks	
OS-level	OS kernel	Containers	
Desktop	User interface	Remote desktops	
Application	Apps	Isolated execution	

3.5 VM Management

1. What is VM Management?

VM Management =

Managing the complete life cycle and resources of Virtual Machines (VMs)

It includes:

- Creating VMs
- Running and monitoring them
- Allocating resources
- Migrating or deleting them

Think:

“How cloud providers control thousands of virtual machines efficiently”

• Virtual Machine lifecycle

A VM goes through different states:

Stages of VM Lifecycle

1. Creation (Provisioning)

- VM is created with:
 - OS
 - CPU (vCPU)
 - Memory
 - Storage

2. Running

- VM is actively executing applications

3. Paused / Suspended

- VM is temporarily stopped
- State is saved in memory/disk

4. Stopped

- VM is shut down

5. Migration

- VM moved to another physical machine

6. Termination

- VM is deleted permanently

• VM provisioning

What it means

Allocating resources to create a VM

Types

- **Static Provisioning**
 - Fixed resources (CPU, RAM)
- **Dynamic Provisioning**
 - Resources adjusted based on demand

Problems

- Over-provisioning → waste
- Under-provisioning → poor performance

• VM placement

What it means

Deciding:

“Which physical server should run this VM?”

Goal

- Balance load
- Use resources efficiently

Factors

- CPU usage
- Memory availability
- Network load

Simple Example

- Server A is overloaded → place VM on Server B

• VM Scheduling

What it means

- Deciding when and how much CPU time a VM gets

How it works

- Hypervisor acts like CPU scheduler
- Uses techniques like:
 - Time sharing
 - Priority scheduling

• VM live migration

What it means

Moving a VM from one physical machine to another

Types of Migration

1. Live Migration

- VM keeps running during migration
- No downtime

How it works

1. Copy memory pages to new server
2. Sync remaining changes
3. Switch execution

2. Cold Migration

- VM is stopped first
- Then moved

Why Migration is Needed

- Load balancing
- Fault tolerance
- Maintenance

• Load Balancing in VM Management

What it means

Distributing VMs across servers

Goal

- Avoid overload
- Improve performance

Algorithms

- Round Robin
- Least Connections

• Resource Allocation in VMs

Resources Managed

- CPU (vCPU)
- Memory (RAM)
- Storage
- Network bandwidth

Key Concepts

- Resource pooling
- Elastic scaling

- **VM Monitoring**

What it means

Tracking VM performance

Metrics

- CPU usage
- Memory usage
- Network traffic
- Disk I/O

Purpose

- Detect overload
- Trigger scaling or migration

- **Fault Tolerance and High Availability**

Fault Tolerance

- VM continues even if hardware fails

High Availability

- VM restarts quickly on another server

Techniques

- Replication
- Checkpointing

- **Snapshot and Cloning**

Snapshot

- Save current state of VM
- Used for rollback

Cloning

- Create identical copy of VM

3.6 Containerization

1. What is Containerization?

Containerization = OS-level virtualization

It allows you to:

- Run multiple applications in **isolated environments (containers)**
- On the **same operating system kernel**

Think:

“Lightweight VMs without separate OS”

- **Virtual Machines vs Containers**

In Virtual Machines:

- Each VM has its **own OS**

In Containers:

- All containers **share the same OS kernel**
- Only apps + dependencies are isolated

Feature	Containers	Virtual Machines
OS	Shared	Separate OS
Size	Small	Large
Boot time	Seconds	Minutes
Performance	Near native	Slight overhead
Isolation	Process-level	Hardware-level
Resource usage	Low	High

One-Line Intuition

- VM → "Virtualizing hardware"
- Container → "Virtualizing OS"

• Architecture of Containerization

Flow

Application → Container → Container Engine → OS → Hardware

Main Components

1. Container

- Package of:
 - Application
 - Libraries
 - Dependencies

2. Container Engine

- Software that runs containers
- Example: Docker

3. OS Kernel

- Shared by all containers

• Working

Containers use Linux kernel features:

1. Namespaces (Isolation)

Namespaces ensure each container:

- Cannot see others

Types:

- Process namespace → separate processes
- Network namespace → separate IP
- Mount namespace → separate file system

Think:

“Each container lives in its own world”

2. Control Groups (cgroups) (Resource Control)

Controls:

- CPU usage
- Memory usage
- Disk I/O

Think:

“Limit how much resource each container can use”

3. Union File System (Layering)

- Container images are built in layers
- Only changes are stored

Example:

- Base OS layer
- App layer
- Update layer

Think:

“Reuse layers → saves storage and speeds up deployment”

5. Container Lifecycle

1. Build
 - Create container image
2. Pull
 - Download image
3. Run
 - Start container
4. Stop
 - Pause execution
5. Remove
 - Delete container

• Advantages

- Lightweight
- Fast startup
- Efficient resource usage
- Easy scaling
- Portable (run anywhere)

• Limitations

- Less isolation than VMs
- OS dependency (same kernel required)
- Security concerns (shared kernel)

• Docker basics

Docker is an open platform that uses OS-level virtualization (containerization) to build, package, and run applications in isolated, lightweight environments called containers. It ensures consistency across different computing environments, solving the common "it works on my machine" problem. [🔗 A Docker Tutorial for Beginners +2](#)

Core Concepts

- **Image:** A read-only template with instructions for creating a Docker container. It contains the application code, runtime, libraries, and dependencies needed to run an application. Images are often built from a plain-text file called a `Dockerfile`.
- **Container:** A runnable instance of an image. Containers are isolated from each other and the host machine but share the host operating system's kernel, making them much more efficient and faster to start than traditional virtual machines.
- **Dockerfile:** A text file that contains all the commands a user could call on the command line to assemble an image.
- **Docker Engine:** The core component (daemon) that runs on the host machine, listening for API requests and managing Docker objects like images, containers, networks, and volumes.
- **Docker Hub/Registry:** A central repository for storing, sharing, and downloading Docker images. Docker Hub is the default public registry.
- **Docker Compose:** A tool for defining and running multi-container applications using a single YAML configuration file.
- **Volumes:** A mechanism for persisting data outside a container's filesystem so that data isn't lost when the container stops or is deleted. [🔗](#)

Key Benefits

- **Consistency:** Guarantees applications run the same way in development, testing, and production environments.
- **Portability:** Containers can run on any system that has Docker installed, from a developer's laptop to a cloud provider.
- **Efficiency:** Containers are lightweight and use system resources more efficiently than virtual machines because they share the host OS kernel.
- **Scalability:** Makes it easy to scale applications by quickly spinning up or tearing down new containers as needed. [🔗 A Docker Tutorial for Beginners +3](#)

Basic Workflow & Commands

1. **Install Docker:** Download and install [Docker Desktop](#) for Windows, Mac, or Linux.
2. **Define Environment (Dockerfile):** Create a `Dockerfile` that specifies the base image and steps to install dependencies and run your application.
3. **Build Image:** Use the command `docker build` to create an image from your Dockerfile.
4. **Run Container:** Use the command `docker run` to start a container from your image. You can use flags like `-d` (detach) and `-p` (publish ports). [🌐 Docker +4](#)

Common Commands:

- `docker pull [image_name]` : Downloads an image from a registry.
- `docker images` : Lists all local images.
- `docker ps` : Lists running containers (use `docker ps -a` to see all, including stopped ones).
- `docker stop [container_id/name]` : Stops a running container.
- `docker rm [container_id/name]` : Removes a stopped container. [🌐 A Docker Tutorial for Beginners +1](#)

• Container orchestration

When many containers run together, you need management.

What it does

- Auto scaling
- Load balancing
- Fault recovery

Example:

- Kubernetes

10. Real-World Flow (Simple Example)

1. Developer builds app → creates container image
 2. Image stored in registry
 3. Cloud system runs containers
 4. Kubernetes manages scaling
-

UNIT 4: Cloud Architecture and Infrastructure

4.1 Data Center Architecture

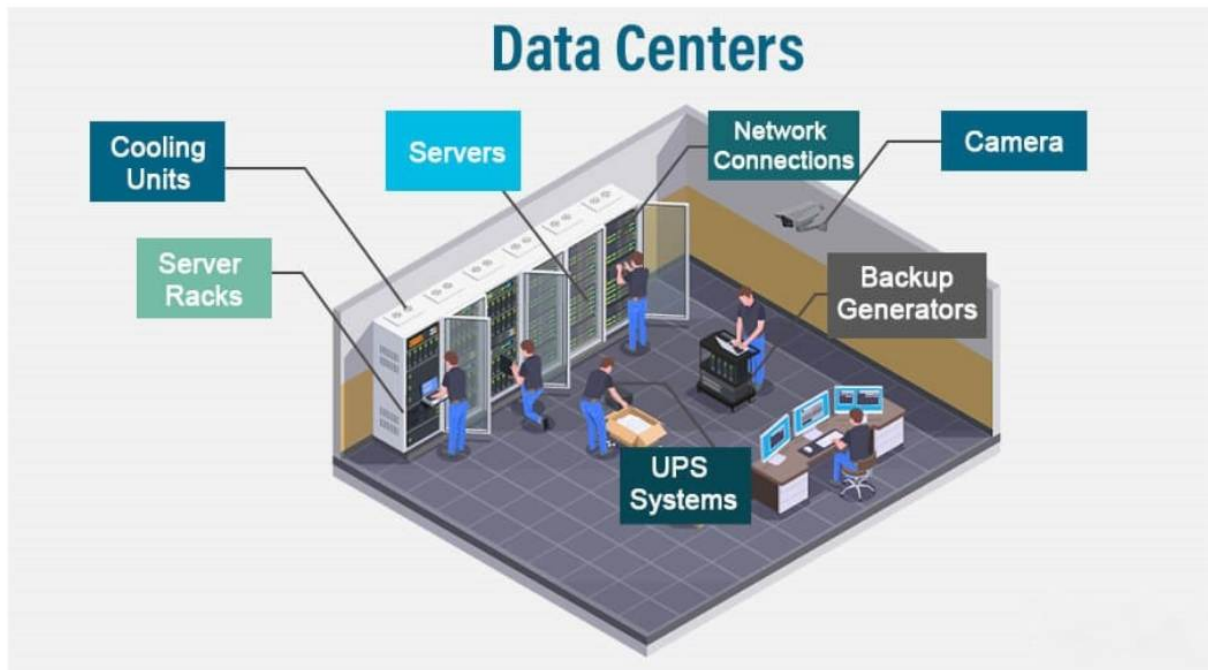
Data centers are physical computing resources that allow organizations to operate their websites or digital offerings 24/7. Data centers are generally made up of racks (servers are stacked with each other), cabinets, cables, and many more. Maintaining a [data center](#) requires a significant amount of networking knowledge. We can host our servers in these data centers either shared or dedicated. Hosted website speed in data centers is usually based on the hardware and specifications of an SSD-based server is faster and more expensive than an HDD-based server.

Data Center: In a dedicated space with strong security levels, where enterprises or organizations store and share large amounts of data, is known as a data center.

Key Aspects of Cloud Data Centers

- **Scalability and Flexibility:** Resources can be instantly scaled up or down based on demand, eliminating the need for over-provisioning infrastructure, according to [GeeksforGeeks](#) and [CloudXon](#).
- **Redundancy and Reliability:** Designed for high availability, these facilities use redundant power feeds, UPS systems, and multiple cooling paths, often reaching Tier III or IV standards (99.982% to 99.995% uptime) to ensure uninterrupted service.
- **Cost-Efficiency (OpEx Model):** Users adopt a pay-as-you-go model, avoiding large upfront capital investments (CapEx) in physical hardware, servers, and cooling systems.
- **Security and Compliance:** Robust security measures include 24/7 physical surveillance, biometric access controls, and cybersecurity measures (firewalls, encryption), with compliance for standards such as ISO 27001, HIPAA, and GDPR.
- **Virtualization and Automation:** Physical servers are heavily virtualized, allowing for efficient resource management and rapid deployment of virtual machines, storage, and databases.
- **Location and Performance:** Data centers are strategically located in multiple geographic regions, allowing users to choose sites closer to their users to minimize latency, as noted in [Amnic](#) and [Atlas Systems](#).
- **Efficiency and Sustainability:** Modern data centers focus on reducing environmental footprints through optimized cooling, energy-efficient designs, and AI-driven workload management, as shown in [GeeksforGeeks](#) and [ESDS](#).  Instaclustr +7

- **Components of data centers**



Core Components of a Data Center:

- **Computing Resources (Servers):** These are the core units that store and process data, often in the form of rack servers, blade servers, and dense server systems.
- **Storage Systems:** Various storage technologies, including DAS, NAS, and SAN, are utilized to securely and efficiently store data.
- **Networking Infrastructure:** This includes routers, switches, and load balancers, acting as the data center's backbone.
- **Power Infrastructure:** This infrastructure involves an uninterruptible power supply (UPS), backup generators, power distribution units (PDUs), and data cabling.
- **Cooling Systems:** To prevent the equipment from overheating, cooling systems consist of computer room air conditioning (CRAC) and computer room air handler (CRAH) units.
- **Physical Security Systems:** Data centers feature robust security measures such as biometric scanners, electronic door locks, and alarm systems.
- **Management Tools:** Data Center Infrastructure Management (DCIM) software is used to monitor and manage all physical and IT infrastructure. 🐦 Sunbird DCIM +3



**Rack
Server**



**Blade
Server**



**Tower
Server**

Physical Infrastructure Features:

- **Racks/Cabinets:** These are used to store all IT equipment, including servers, in an organized manner, helping to maximize space.
- **Cable Management:** High-density cable management and structured cabling systems help to maintain order and performance, ensuring easy troubleshooting and management of network and power cables.
- **Raised Floors:** They help in efficient cooling and for cable management.
- **Backup Systems:** Redundant systems for power and cooling are essential to maintain operations in case of component failure.
- **Security Measures:** Physical security systems include biometric scanners, card readers, and video surveillance to protect the infrastructure.
- **Fire Suppression:** Specialized systems designed for data center environments to protect against fire without damaging equipment.  Sunbird DCIM +4

Data Center Ecosystem Types:

- **Hyperscale Data Centers:** These are large, purpose-built, and designed for massive-scale cloud computing, often serving 100 megawatts or more of capacity.
- **Colocation Facilities:** These are data centers where companies can rent space for their servers and other equipment, benefiting from shared, secure infrastructure.  Fortinet +1

Modern Data Center Trends:

- **Cloud Integration:** Hybrid and multi-cloud strategies are becoming more common.
- **AI-Driven Optimization:** Automation and artificial intelligence are being used to manage data centers more efficiently and improve operations.
- **Sustainability:** Energy efficiency is a critical focus, with organizations aiming to reduce the power consumption of their data centers.
- **Redundancy Models:** These are used to increase the fault tolerance of the system.  Atlas Systems +2

• Design considerations

Key design goals -

1. High Scalability and Elasticity

- **Modular Growth:** Designing with a modular approach (e.g., containerized data centers) allows for the addition of power, cooling, and compute resources as needed without disrupting existing operations.
- **Elastic Resources:** Infrastructure is designed to scale up or down based on rapid changes in demand, ensuring that cloud providers can accommodate fluctuating workloads.
- **Horizontal Scaling:** Emphasizing the ability to add more servers to the network rather than just increasing the power of a single server (vertical scaling).  Digital Realty +3

2. High Availability and Reliability (Redundancy)

- **Zero Single Point of Failure:** Designing systems with redundant power (N+1, 2N), cooling, and network paths to ensure continuous uptime.
- **Fault Tolerance:** Implementing automated failover systems so that if one component fails, the workload is automatically shifted to a backup without service interruption.
- **Geographic Redundancy:** Distributing infrastructure across multiple locations to protect against regional disasters.  Oracle +4

3. Energy Efficiency and Sustainability

- **Low PUE (Power Usage Effectiveness):** A primary goal is to bring PUE as close to 1.0 as possible, minimizing energy wasted on non-IT operations.
- **Advanced Cooling Techniques:** Utilizing hot/cold aisle containment, liquid cooling, or immersion cooling to handle high-density server heat efficiently.
- **Renewable Energy Integration:** Powering facilities with solar, wind, or other renewable sources to meet corporate ESG goals. ● Bridgepointe Technologies +2

ESG (Environmental, Social, and Governance)

4. High Performance and Low Latency

- **Leaf-Spine Topology:** Using a leaf-spine network design to improve traffic management and reduce latency for data processing between servers.
- **Edge Computing Integration:** Placing data center resources closer to end-users to reduce data travel time, crucial for 5G and IoT applications.
- **High-Speed Interconnects:** Employing 100G and higher networking technology to support massive data throughput. 🚀 gbc engineers +3

5. Security and Regulatory Compliance

- **Layered Security (Physical and Digital):** Combining perimeter fencing and biometric access controls with advanced software-defined security (firewalls, encryption, zero-trust).
- **Data Sovereignty:** Designing to comply with specific regional regulations (e.g., GDPR) concerning where data is stored and accessed. 🌐 Oracle +2

General Data Protection Regulation
Data privacy law

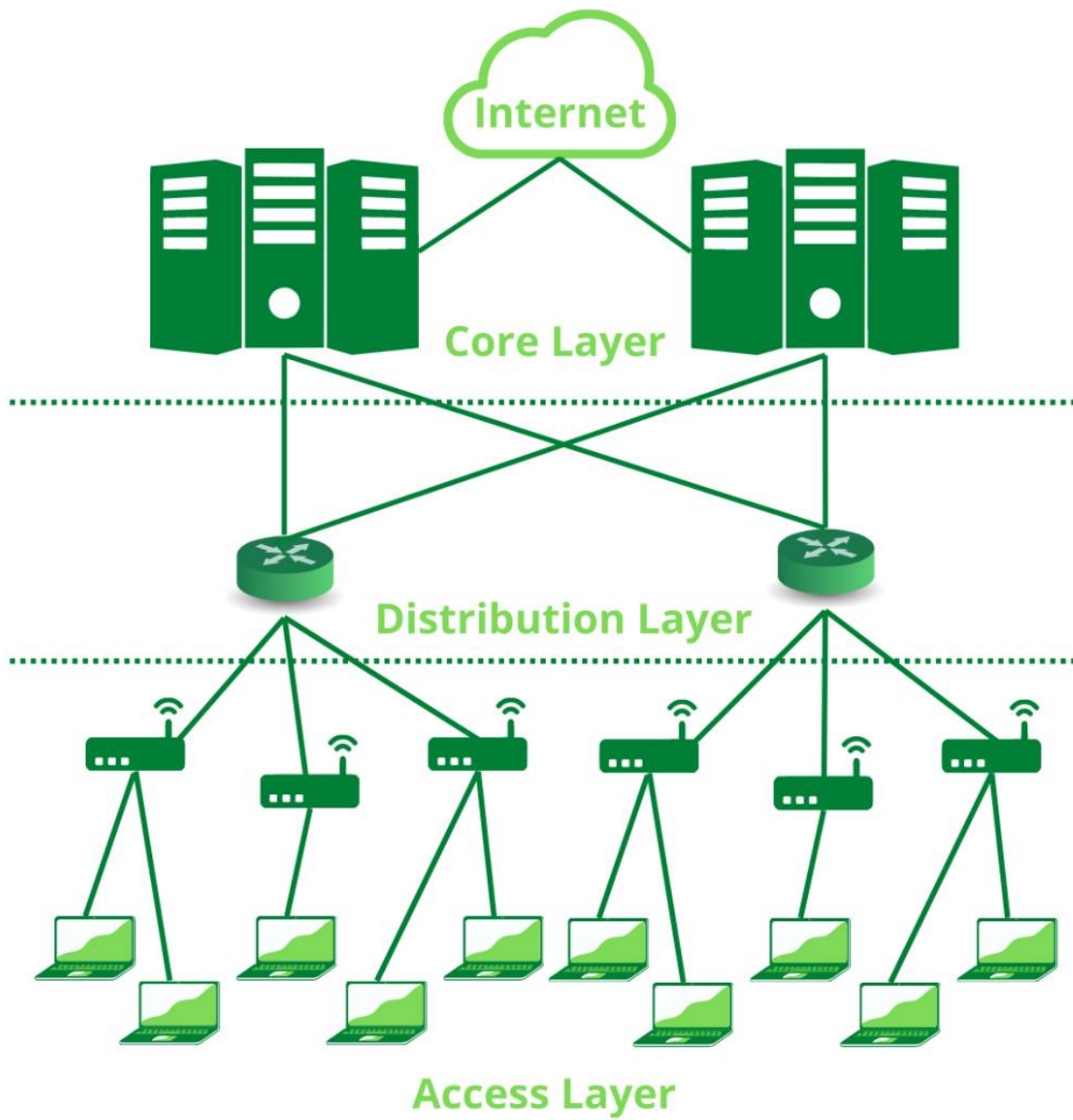
6. Operational Efficiency and Automation

- **Software-Defined Everything:** Utilizing Software-Defined Networking (SDN) and Data Center Infrastructure Management (DCIM) tools to enable automated, rapid provisioning of resources.
- **AI-Powered Optimization:** Implementing AI algorithms to predict hardware failures and optimize cooling based on real-time loads. 🤖 Atlas Systems +2

7. Cost-Effectiveness

- **Total Cost of Ownership (TCO) Optimization:** Balancing initial capital expenditure (CapEx) with long-term operating expenses (OpEx) to ensure financial viability.
- **Virtualization:** Maximizing hardware utilization by running multiple virtual machines on a single physical server. 🔄 Flexential +2

Traditional 3-layer architecture



The 3-layer data center network architecture consists of **Core, Aggregation/Distribution, and Access layers**, providing a hierarchical, reliable structure for traffic management. It facilitates high-speed routing to the internet/core, aggregates traffic, and connects thousands of servers via access switches. [Park Place Technolog...](#) +2



Key Layers of 3-Tier Data Center Architecture

- **Access Layer (Edge):**
 - **Function:** Connects end devices like servers, storage, and networking hardware to the network, [as explained in this YouTube video](#).
 - **Components:** Top-of-rack (ToR) switches providing 1/10Gbps connectivity, [as described in the World Wide Technology | WWT article](#).
 - **Features:** Often implements Layer 2 switching and connects to the aggregation layer.
- **Aggregation/Distribution Layer:**
 - **Function:** Aggregates data from multiple access layer switches and provides policy-based connectivity, security (firewalling), and VLAN routing, as described in the World Wide Technology | WWT article.
 - **Features:** High-speed switching, providing redundancy and managing the traffic from the access layer.
- **Core Layer (Backbone):** [fast transport between aggregation switches and to external networks](#)
 - **Function:** Provides high-speed, reliable connectivity between different sections of the data center, connecting multiple aggregation switches.
 - **Components:** Large, high-performance modular chassis switches.
 - **Features:** Focuses on maximum throughput, speed, and reliability to move traffic efficiently. [World Wide Technolo...](#) +5

Key Benefits

- **Hierarchical Management:** [Simplifies troubleshooting and network expansion](#), as explained in this YouTube video.
- **Scalability:** Allows adding more access switches and racks without disrupting the entire structure.
- **Traffic Management:** Efficiently routes traffic between access and core layers, according to [Park Place Technologies](#). [Park Place Technolog...](#) +4

While effective, this traditional 3-tier architecture can create bottlenecks at the core layer, prompting some organizations to move toward a flatter **Spine-Leaf** architecture for high-performance computing, as noted by Park Place Technologies. [Park Place Technologies](#)

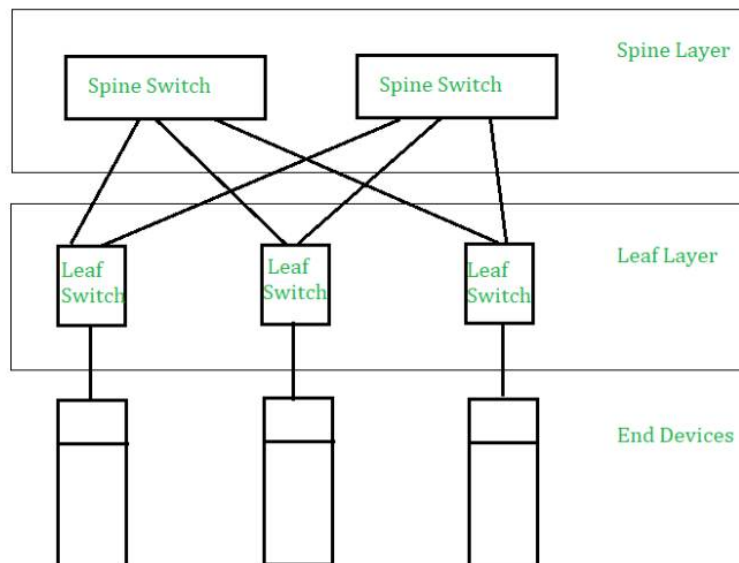
Key Limitations:

- **Poor East-West Performance:** Data centers today mostly handle server-to-server (east-west) traffic, but 3-tier models are optimized for client-to-server (north-south), causing high latency and bottlenecks.
- **Limited Scalability:** As network needs grow, adding more switches leads to a "full-mesh," scenario that becomes impossible to scale efficiently, often requiring a total redesign.
- **Bottlenecks and Oversubscription:** High traffic loads frequently create bottlenecks in the aggregation or core layers, reducing overall network performance.
- **High Complexity and Cost:** Implementing and managing multiple distinct layers, along with complicated STP (Spanning Tree Protocol) settings for redundancy, increases both operating expenses (CapEx/OpEx) and management overhead.
- **Lack of Flexibility:** The hierarchical, static nature of the 3-tier model makes it less adaptable to dynamic, virtualized, or cloud-native applications.
- **Spanning Tree Issues:** To prevent network loops, this architecture often blocks redundant paths, meaning you pay for infrastructure that cannot be used for traffic forwarding. [Park Place Technolog...](#) +5

Modern architectures, such as **Spine-Leaf**, address these issues by providing a flatter, faster topology that ensures low-latency and equal-cost multi-pathing (ECMP) for better performance. [Networkers Home](#) +1

Spine leaf architecture - 2-layer architecture

<https://www.youtube.com/watch?v=xjc7WLBb-nI>



Spine-leaf architecture is a modern two-layer data center network topology consisting of spine and leaf switches, designed to optimize east-west traffic flow and provide high-bandwidth, low-latency connectivity. It replaces traditional three-tier architectures, offering superior scalability, predictable performance, and high redundancy through a full-mesh topology. [Hewlett Packard Ente... +2](#)



Core Components

- **Leaf Layer:** Comprises access switches that connect to servers, storage, and other network devices. Leaf switches are "top-of-rack" and connect directly to all spine switches.
- **Spine Layer:** Acts as the backbone of the network, connecting all leaf switches. Spine switches do not connect to each other directly, ensuring all traffic passes through them from leaf to leaf. [YouTube +2](#)

Key Benefits

- **Improved Performance:** Predictable, low-latency communication because any server is always only two hops away from another server.
- **High Scalability:** Capacity can be expanded by adding more spine switches (for more bandwidth) or leaf switches (for more server connections).
- **High Redundancy & Reliability:** Because every leaf is connected to every spine, the failure of a single spine switch only reduces bandwidth rather than causing a complete network outage.
- **East-West Traffic Optimization:** Specifically designed for modern data centers where servers frequently communicate with each other (virtual machine migration, storage traffic) rather than just with the outside world. [Hewlett Packard Ente... +4](#)

Spine-Leaf vs. Traditional 3-Tier Architecture

Feature	Traditional 3-Tier	Spine-Leaf
Layers	Core, Aggregation, Access	Leaf, Spine
Main Traffic	North-South (client-to-server)	East-West (server-to-server)
Scalability	Complex, limited by STP	Easy, by adding switches
Latency	Higher, unpredictable	Low, predictable
Loop Handling	Spanning Tree Protocol (STP)	Layer 3 Routing (ECMP)

How It Works

The architecture operates on a full-mesh, non-blocking fabric, meaning every leaf switch connects to every spine switch. It relies on Layer 3 routing (like ECMP—Equal-Cost Multi-Pathing) to spread traffic across all available paths, eliminating the bottlenecks common in legacy spanning-tree environments. While primarily Layer 3, virtualization techniques such as VXLAN can provide Layer 2 connectivity over the top. [YouTube +3](#)

4.2 Distributed Architecture

• Distributed systems basics

Definition

A distributed system is a collection of **multiple computers (nodes)** that work together as a **single system**

Key Characteristics

- Multiple nodes
- Communication via network
- No shared memory
- Appears as one system

Example

- Web application:
 - Frontend server
 - Backend server
 - Database server

• Resource pooling

Definition

Combining resources (CPU, memory, storage) and sharing them among multiple users

How it works

- Cloud provider creates a **pool of resources**
- Dynamically allocates to users

Example

- Many users share same data center servers

Key Benefit

- Efficient utilization
- Cost reduction

4.3 Reliability and Performance

• Fault tolerance

Definition

System continues to work even if some components fail

Techniques

- Replication (multiple copies)
- Redundancy
- Checkpointing

Example

- If one server fails → another takes over

• High availability

Definition

System is **available almost all the time**

Measured as

- Uptime percentage:
 - 99.9% (3 nines)
 - 99.99% (4 nines)

Example

- Cloud services running 24/7

- **Disaster recovery**

Definition

Recovering system after **major failure** (e.g., data center crash)

Techniques

- Backup
- Data replication across regions
- Failover systems

Example

- Data center in one city fails → switch to another

Difference (Very Important)

Concept	Meaning
Fault Tolerance	Handle small failures
High Availability	Minimize downtime
Disaster Recovery	Recover from major failures

4.4 Load Balancing

Need for Load Balancing

Why needed

- Avoid server overload
 - Improve performance
 - Ensure availability
-

Types

1. Hardware Load Balancer

- Physical device
 - High performance
 - Expensive
-

2. Software Load Balancer

- Runs as software
- Flexible
- Used in cloud

Algorithms

1. Round Robin

Requests distributed one by one

$A \rightarrow B \rightarrow C \rightarrow A \rightarrow B \rightarrow C$

2. Least Connections

Send request to server with **least active connections**

Key Benefit

- Balanced resource usage
- Faster response time

4.5 Architectural Models

- **N-tier architecture**

N-tier Architecture

Definition

Application divided into **multiple layers (tiers)**

Common Types

1. 2-Tier

- Client ↔ Database

2. 3-Tier (Most Important)

- Presentation layer (UI)
- Application layer (logic)
- Data layer (database)

Example

User

↓

Web Server (UI)

↓

Application Server

↓

Database

Advantages

- Scalability
- Maintainability
- Security



4.6 Agreements and Metrics

- **Service Level Agreements (SLA)**

Definition

A contract between provider and user defining **service expectations**

Includes

- Availability (uptime %)
- Performance
- Response time
- Support

Example

- 99.9% uptime guarantee

• Performance metrics

Definition

Measures used to evaluate system performance

Important Metrics

1. Latency

- Time to respond
-

2. Throughput

- Number of requests processed per second
-

3. Response Time

- Total time taken to handle request
-

4. Availability

- Percentage of uptime
-

Example

- Website response time = 200 ms
-

UNIT 5: Cloud Storage and Data Management

5.1 Storage Models

• Block Storage

Definition

Data is stored in **fixed-size blocks**, each with a unique address

How it works

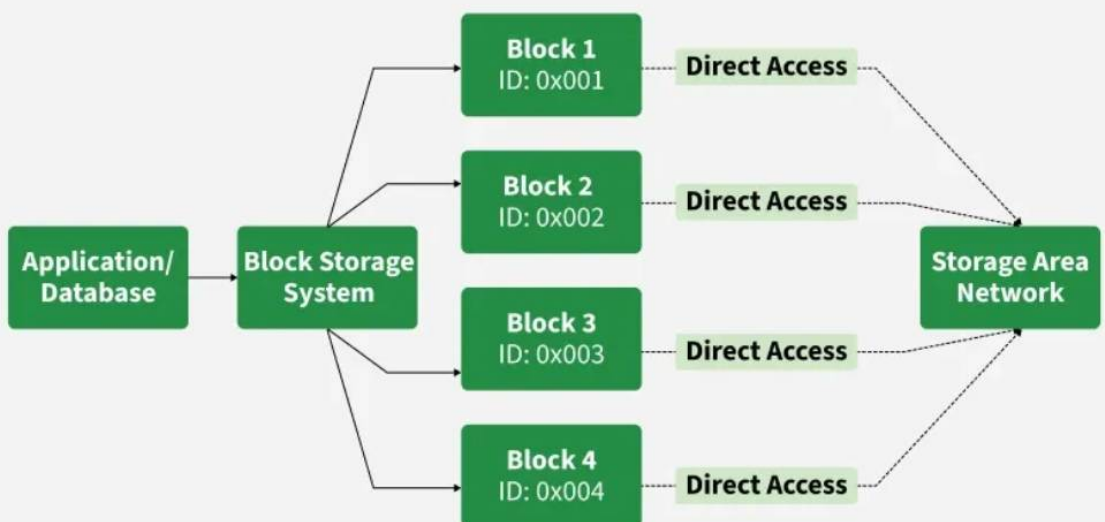
- Data → split into blocks
- Each block stored separately
- OS manages how blocks are combined

Example Use

- Virtual machine disks
- Databases

Key Features

- High performance
- Low latency
- Flexible



• File Storage

Definition

Data stored as **files** in a hierarchical structure

Structure

- Folders → Subfolders → Files

Example Use

- Shared file systems
- Network drives

Key Features

- Easy to use
- Human-readable organization

• Object Storage

Definition

Data stored as **objects** with metadata and unique ID

Structure

- Object = Data + Metadata + Unique ID

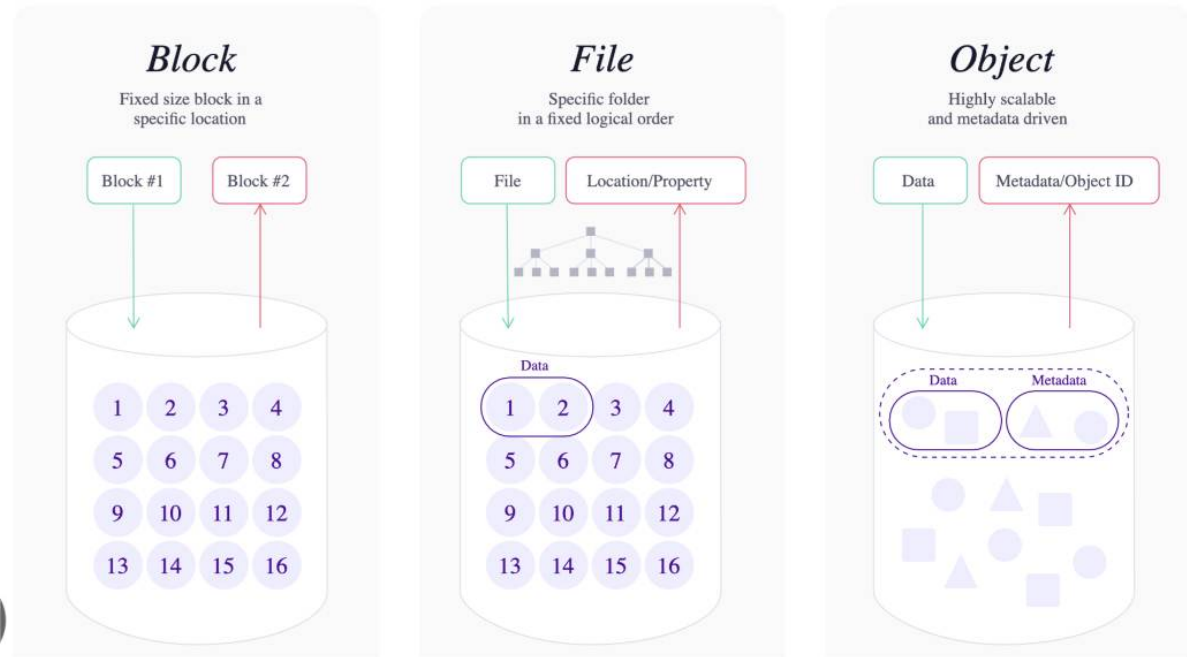
Example Use

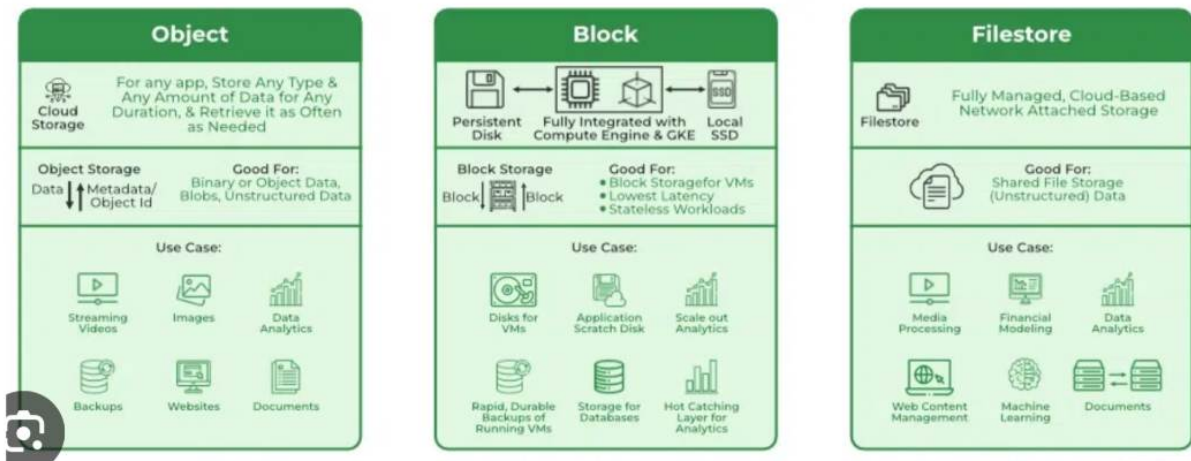
- Images, videos, backups

Key Features

- Highly scalable
- Used in cloud (like S3-type systems)

Feature 	Block Storage	File Storage	Object Storage
Data Structure	Raw Blocks (<u>fixed size</u>)	Files & Folders (<u>Hierarchical</u>)	Flat (<u>Object + Metadata</u>)
<u>Performance</u>	Very High/Low Latency 	Moderate (files), High (NAS)	Lower (high latency)
<u>Scalability</u>	Limited	Moderate/Limited	Massive/ <u>Unlimited</u> 
Access Method	<u>SAN/OS-mounted Volume</u>	<u>NAS/Shared File System</u>	<u>API/RESTful</u>
Best For	<u>DBs</u> , <u>VMs</u> , Apps	NAS, Sharing, Home Dirs	<u>Data Lakes</u> , Backup, Cloud
Cost	High	Moderate/High	Low 





5.2 Distributed File Systems

• HDFS (Hadoop Distributed File System)

HDFS (Hadoop Distributed File System)

Definition

A distributed file system designed to store large data across multiple machines

Architecture

1. NameNode (Master)

Role

- Stores metadata:
 - File names
 - Block locations

Important

- Does NOT store actual data

2. DataNode (Worker)

Role

- Stores actual data blocks

3. Replication (Very Important)

Definition

Storing multiple copies of data blocks



Default

- 3 copies of each block

Purpose

- Fault tolerance
- Data reliability

Working Example

1. File uploaded
2. Split into blocks
3. Stored in multiple DataNodes
4. NameNode tracks locations

• Google File System (GFS)

Similar to HDFS

- Master node (metadata)
- Chunk servers (data storage)

Key Idea

- Large-scale distributed storage

5.3 Database Systems

• RDBMS in cloud

Definition

Traditional relational databases hosted in cloud

Features

- Tables (rows + columns)
- SQL queries
- ACID properties

Example Use

- Banking systems
- Enterprise applications

• NoSQL databases:

Why NoSQL?

- Handle:
 - Large data
 - Unstructured data
 - High scalability

Types of NoSQL

1. Key-Value Store

- Data stored as:
 - Key → Value

Example:

- "user1" → data

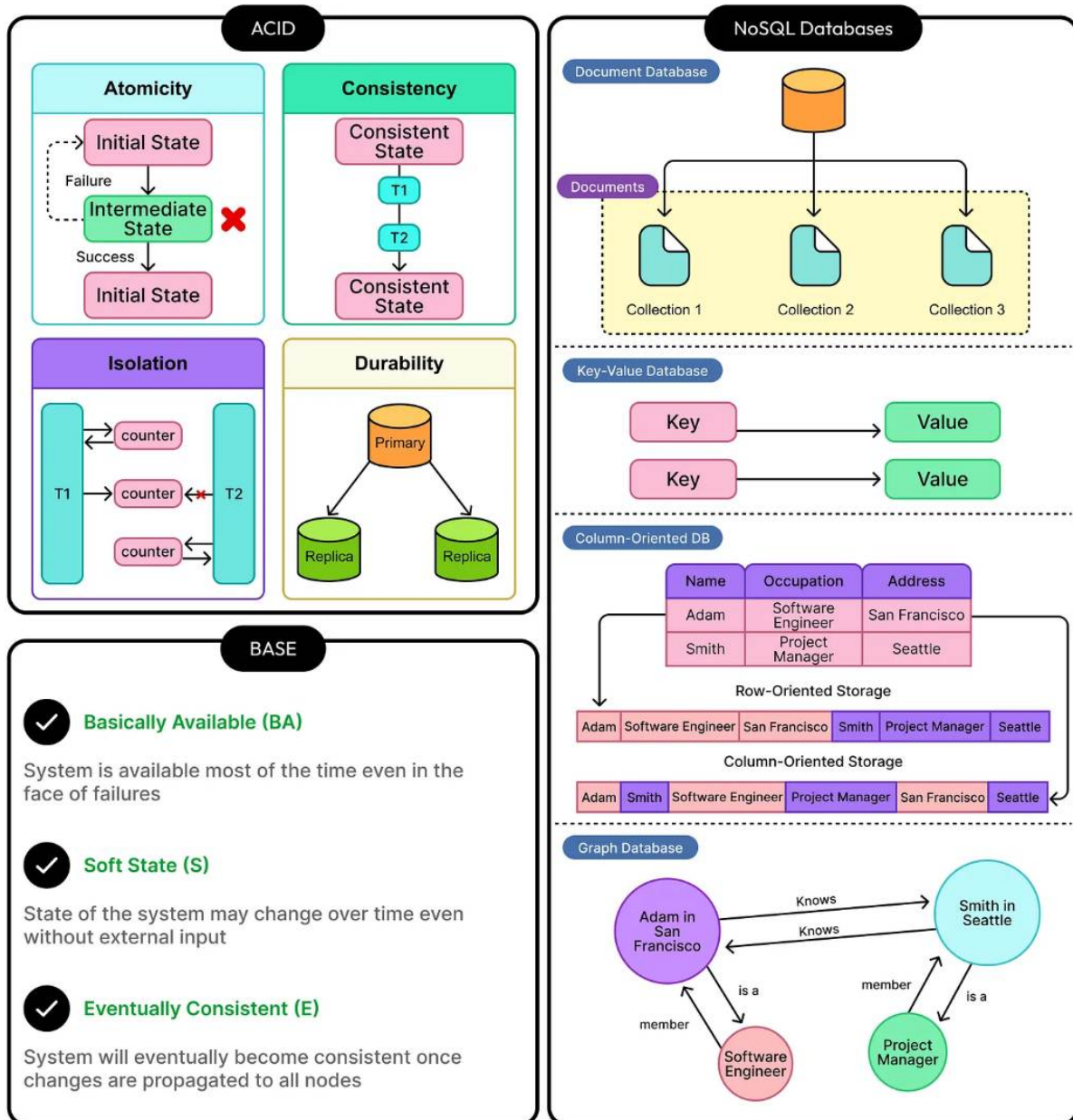
2. Column-Oriented

- Data stored in columns instead of rows

3. Document-Based

- Data stored as JSON-like documents

SQL vs NoSQL: Choosing the Right Database



5.4 Data Consistency and Models

• CAP Theorem

The CAP theorem states that **a distributed database system can only provide two of three guarantees: Consistency (all nodes see the same data), Availability (every request gets a response), and Partition Tolerance (system operates despite network failures)**. In the presence of a network partition, a system must choose between consistency or availability.  IBM +3

Key Components of CAP Theorem:

- **Consistency (C):** Every read receives the most recent write or an error. All clients see the same data at the same time.
- **Availability (A):** Every request receives a (non-error) response, without the guarantee that it contains the most recent write. The system remains operational.
- **Partition Tolerance (P):** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.  Wikipedia +2

Trade-offs (The Choice):

- **CP (Consistency + Partition Tolerance):** The system prioritizes data accuracy. If a network partition occurs, the system will return an error or time out, sacrificing availability to ensure all nodes remain consistent. Example: Distributed databases (e.g., MongoDB, [HBase](#)).
- **AP (Availability + Partition Tolerance):** The system prioritizes uptime. If a partition occurs, nodes may return stale or conflicting data, but the system remains reachable. Example: Cassandra, DynamoDB.
- **CA (Consistency + Availability):** While possible in theory, a system that cannot tolerate network partitions (no P) is not practical in distributed environments.  GeeksforGeeks +4

Real-World Application:

The CAP theorem is essential for designing distributed systems. [Financial systems](#) usually prioritize **CP** to ensure data accuracy, while [social media feeds or caching systems](#) often prioritize **AP** for high responsiveness.  AlgoMaster Newsletter +3

• BASE properties

The **BASE** properties (Basically **Available, Soft State, Eventual Consistency**) are a set of principles used in modern, distributed database systems (primarily NoSQL databases) that prioritize availability and scalability over immediate data consistency, contrasting with the traditional ACID properties. [Amazon Web Services \(AWS\) +1](#)

The acronym BASE stands for:

- **Basically Available:** The system guarantees availability, meaning it is accessible to users most of the time, even during partial system failures or network outages. The system ensures that a user request receives a response, although the data might not be the most recent version.
- **Soft State:** The state of the system can change over time, even without any explicit user input, due to the nature of distributed systems and the propagation of changes. Data can exist in a temporary or transient state until all updates are synchronized across the system. Responsibility for handling these transient states often falls on the developer.
- **Eventual Consistency:** The system guarantees that if no new updates are made to a specific data item, all accesses to that data item will eventually return the last written value. Consistency is achieved over time after a propagation delay, rather than immediately after a transaction commits. [GeeksforGeeks +4](#)

Key Differences from ACID

Feature 🔗	ACID (Traditional DBMS)	BASE (NoSQL/Distributed Systems)
Priority	Prioritizes consistency over availability.	Prioritizes availability over consistency.
Consistency	Requires immediate and strict consistency for all transactions.	Only requires eventual consistency, allowing temporary inconsistencies.
Scaling	Typically scales vertically (harder to scale horizontally).	Designed to scale horizontally across multiple nodes.
Use Cases	Suitable for systems requiring high data integrity (e.g., banking, financial systems).	Suitable for highly available, large-scale systems (e.g., e-commerce, social media platforms).

The choice between ACID and BASE properties depends on the specific requirements of the application, as the CAP theorem states that a distributed system can only fully guarantee two out of the three properties: Consistency, Availability, and Partition Tolerance. [Amazon Web Services \(AWS\)](#)

• Consistency models

1. Strong Consistency (Linearizability)

- **Definition:** All nodes see the same data at the same time. A read always returns the latest write, ensuring immediate consistency.
- **Where it's seen:** Financial systems, inventory management, or applications requiring strict data integrity.
- **Example:** A banking app. If a user transfers \$100, the deduction from account A and addition to account B must be immediately visible to everyone (using systems like **Google Spanner** or traditional **SQL** databases).  Milvus +3

2. Eventual Consistency

- **Definition:** Replicas may temporarily diverge, but all nodes will converge to the same value over time if no new updates are made.
- **Where it's seen:** Social media, content delivery networks (CDNs), and high-availability systems.
- **Example:** Social media likes or YouTube view counts. If you like a video, it takes a few seconds to reflect the updated number of likes across all global servers (commonly used in **Apache Cassandra** or **Amazon DynamoDB**).  Decube +4

3. Causal Consistency

- **Definition:** Only related (causal) operations are seen in the same order by all nodes. Independent operations can be seen in different orders.
- **Where it's seen:** Social media comments, chat applications, and collaborative editing.
- **Example:** If User A posts a photo and User B comments, causal consistency ensures everyone sees the photo *before* the comment. A system like **Azure Cosmos DB** can provide this.  Medium +4

4. Read-Your-Writes Consistency

- **Definition:** Guarantees that if a user makes an update, they will immediately see that update in subsequent reads, even if other users do not.
- **Where it's seen:** User dashboards, personalized apps, and profile updates.
- **Example:** Updating a profile picture on Facebook or Instagram. You see the new picture immediately, while your followers might see the old one for a few more seconds. Mk Medium +4

5. Session Consistency

- **Definition:** A form of read-your-writes that guarantees consistency *within* a user's session, but not necessarily across different sessions.
- **Where it's seen:** Shopping carts in e-commerce, or logged-in user activity.
- **Example:** While browsing Amazon, your cart updates instantly, but if another user checked your cart from a different device, they might see a slightly outdated version temporarily. Mk Kousik Nath - Medium

Consistency Model 	Visibility	Performance	Example Application
Strong	Instant	Low (Slow)	Banking (\$)
Eventual	Delayed	High (Fast)	Social Likes
Causal	Ordered	Medium	Chat Apps
Read-Your-Writes	Instant (Self)	Medium	Profile Update
Session	User-Session	High	Shopping Cart

- **Database Types:** Relational databases (e.g., **PostgreSQL**) typically favor Strong Consistency, while NoSQL databases (e.g., **Cassandra**) often default to Eventual Consistency. Mk Medium +2

5.5 Big Data Frameworks

• MapReduce paradigm

MapReduce Paradigm (Very Important)

Definition

Programming model for processing large data in parallel

Two Phases

1. Map Phase

- Input → processed into key-value pairs
-

2. Reduce Phase

- Combine results
-

Example

Count words:

- Map → (word, 1)
 - Reduce → sum counts
-

Key Advantage

- Parallel processing
- Scalable

• Hadoop ecosystem basics

Core Components

- HDFS → storage
 - MapReduce → processing
-

Additional Tools (Basic Awareness)

- Hive (querying)
 - Pig (data processing)
-

UNIT 6: Networking in Cloud

6.1 Cloud Networking Basics

1. Cloud Networking Basics

What is Cloud Networking?

Networking of resources (VMs, storage, services) inside the cloud using **virtual networks** instead of physical cables

Key Idea

Instead of:

- Physical routers
- Physical switches

Cloud uses:

- **Software-defined networking (SDN)**
-

Basic Components

- Virtual machines (instances)
- IP addresses
- Subnets
- Routing tables
- Gateways
- Firewalls (security groups)

Simple Flow

User → Internet → Cloud Network → VM

Important Concepts

- Private IP (internal communication)
- Public IP (internet access)
- Isolation (each user gets separate network)

• Virtual Private Cloud (VPC)

Definition

A VPC is a **logically isolated virtual network** inside a cloud provider

Think:

"Your own private network inside the cloud"

Key Features

- Isolation from other users
- Custom IP range
- Control over:
 - Subnets
 - Routing
 - Security

How VPC Works

1. You create a VPC
2. Assign IP range (CIDR block)
Example:
 - 10.0.0.0/16
3. Inside VPC, you create:
 - Subnets
 - Route tables
 - Gateways



Types of VPC Access

1. Public Access

- Connected to internet via **Internet Gateway**

2. Private Access

- No direct internet access

Key Components of VPC

1. Internet Gateway (IGW)

- Connects VPC to internet

2. NAT Gateway

- Allows private subnet → internet (outgoing only)

3. Route Table

- Defines traffic flow

• Subnets

Definition

A subnet is a **smaller network inside a VPC**

Why Subnets?

- Organize resources
- Improve security
- Control traffic

Types of Subnets

1. Public Subnet

- Has route to internet
- Used for:
 - Web servers

2. Private Subnet

- No direct internet access
- Used for:
 - Databases
 - Backend services

Example

VPC: `10.0.0.0/16`

- Subnet 1: `10.0.1.0/24` (Public)
- Subnet 2: `10.0.2.0/24` (Private)

• Routing in cloud

What is Routing?

Deciding where network traffic should go

Route Table

A route table contains rules like:

Destination	Target
0.0.0.0/0	Internet Gateway
10.0.0.0/16	Local

How Routing Works

Example:

1. VM sends request to internet
 2. Route table checks:
 - If destination = 0.0.0.0/0 → send to IGW
 3. Traffic goes to internet
-

Key Routing Concepts

1. Local Route

- Default route inside VPC
-

2. Internet Route

- Sends traffic outside VPC
-

3. NAT Routing

- Private subnet → NAT → Internet

5. Putting It All Together (Important)

Complete Flow Example

1. Create VPC (10.0.0.0/16)
2. Create subnets:
 - Public subnet (web server)
 - Private subnet (database)
3. Attach Internet Gateway
4. Configure routes:
 - Public subnet → IGW
 - Private subnet → NAT

Architecture View

Internet



Internet Gateway



Public Subnet (Web Server)



Private Subnet (Database)

6. Exam-Focused Key Points

Must remember:

- VPC = isolated cloud network
- Subnet = smaller network inside VPC
- Public subnet → has internet access
- Private subnet → no direct internet
- Route table → controls traffic flow
- NAT → private → internet (outgoing only)

7. One-Line Intuition

- VPC → "Your private network in cloud"
- Subnet → "Rooms inside that network"
- Routing → "Rules that decide where traffic goes"

6.2 Network Components

• DNS

<https://www.youtube.com/watch?v=g6R9gRWIIK8>

Definition

DNS converts a **domain name** (like google.com) into an **IP address** (like 142.250.x.x)

Why DNS is Needed

Humans use:

- Domain names (easy to remember)

Computers use:

- IP addresses

DNS acts like a **translator**

How DNS Works (Step-by-Step)

1. User enters: `www.example.com`
2. Request goes to **DNS resolver**
3. Resolver queries DNS servers
4. Gets IP address
5. Browser connects to that IP

Types of DNS Servers

- Root DNS server
- TLD server (.com, .org)
- Authoritative DNS server

Cloud Relevance

- Maps domain → cloud resources (VM, load balancer, etc.)
- Enables scalability and failover

One-Line Intuition

DNS = "Phonebook of the internet"

• Load balancers

Definition

A load balancer distributes incoming traffic across multiple servers

Why Needed

- Prevents server overload
- Improves performance
- Ensures high availability

How It Works

1. User request comes in
2. Load balancer receives request
3. Chooses a server
4. Forwards request

Types of Load Balancers

1. Layer 4 (Transport Layer)

- Works on IP + port
 - Faster
 - Less intelligent
-

2. Layer 7 (Application Layer)

- Works on HTTP/HTTPS
 - Can route based on:
 - URL
 - Headers
 - More flexible
-

Common Algorithms

- Round Robin
 - Least Connections
 - IP Hash
-

Cloud Use Case

- Multiple web servers behind one load balancer

Example Usage (Layer 4)

Use Case: High-speed API or database traffic

Example:

- A company has:
 - 3 backend servers running APIs
- Requests come on port **443 (HTTPS)**

Flow

1. Request arrives
 2. Load balancer checks:
 - IP + port
 3. Forwards to one of the servers
-

Why Layer 4 here?

- Fast (no deep inspection)
- Suitable for:
 - TCP services
 - Gaming servers
 - Streaming
 - Databases

Layer 7 Load Balancer (Application Layer)

What it uses

- HTTP/HTTPS data
 - URL
 - Headers
 - Cookies
-

How it works

- Understands request content
 - Can route based on:
 - URL path
 - Request type
-

Example Usage (Layer 7)

Use Case: Web Application Routing

Example:

- Website:
 - `/api` → backend servers
 - `/images` → image servers
 - `/login` → auth server

Flow

1. Request: `example.com/api/users`
 2. Load balancer reads URL
 3. Sends to API server
-

Why Layer 7 here?

- Smart routing
- Needed for:
 - Microservices
 - Web apps
 - REST APIs

3. Load Balancing Algorithms (Easy + Technical)

1. Round Robin

Idea

Send requests to servers one by one in order

How it works

If servers = A, B, C

Requests go:

- 1 → A
 - 2 → B
 - 3 → C
 - 4 → A
 - repeat
-

Best for

- Servers with equal capacity

2. Least Connections

Idea

Send request to server with **fewest active connections**

How it works

- Server A → 10 users
 - Server B → 3 users
→ New request goes to **B**
-

Best for

- Uneven load
- Long-running connections

3. IP Hash

Idea

Same client IP always goes to same server

How it works

- Hash(client IP) → choose server

Example:

- User 1 → Server A
- User 2 → Server B
- User 1 again → Server A

Best for

- Session persistence (sticky sessions)

• Content Delivery Networks (CDN)

Definition

A CDN is a network of distributed servers that deliver content from the **nearest location to the user**

Why CDN is Needed

- Reduce latency
- Faster content delivery
- Reduce load on origin server

How It Works

1. User requests content (image/video)
2. CDN checks nearest server (edge server)
3. If cached → serve immediately
4. If not → fetch from origin server
5. Cache it for future

Key Components

1. Edge Servers

- Located globally
- Store cached content

2. Origin Server

- Main server where data is stored

Types of Content

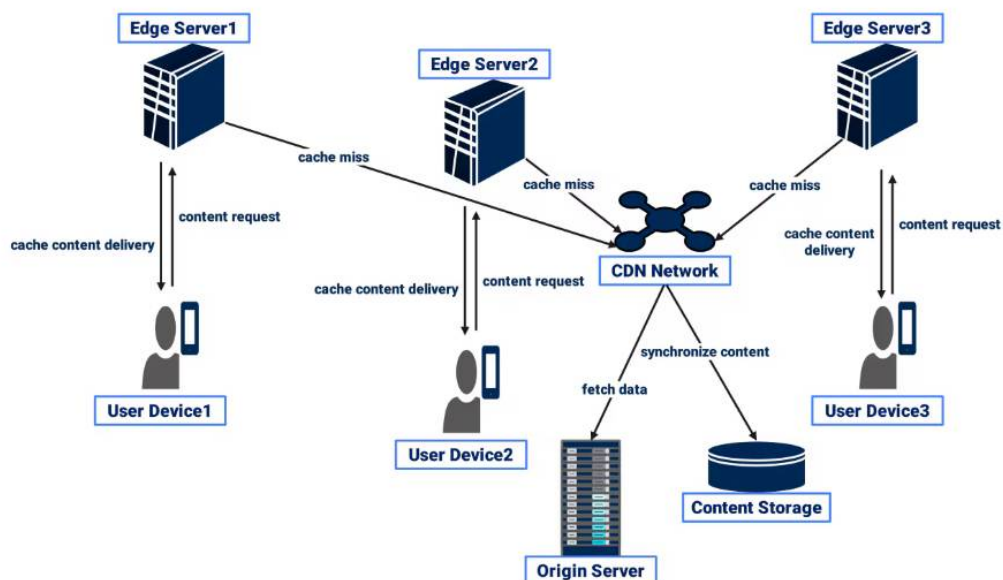
- Static:
 - Images
 - Videos
 - CSS/JS
- Dynamic (advanced CDNs)

Cloud Example

- Websites using global users

One-Line Intuition

CDN = "Serve data from nearest location"



6.3 Advanced Networking

1. Big Picture First (Very Important)

Traditional networking:

- Hardware-based (routers, switches, firewalls)
- Hard to configure and scale

Modern cloud networking:

- **Software-controlled + virtualized**

Key Idea

Concept	Focus
SDN	Control the network using software
NFV	Replace hardware devices with software
Network Virtualization	Create multiple virtual networks

• Software Defined Networking (SDN)

Definition

SDN separates the **control plane** from the **data plane** and manages the network using software

Important Terms

1. Data Plane

- Actually forwards packets

2. Control Plane

- Decides where packets should go

Traditional Network

- Each device:
 - Decides + forwards

SDN Network

- Central controller decides everything
- Devices only forward packets

Architecture

Controller (brain)

↓

Switches/Routers (workers)

How SDN Works

1. Packet arrives at switch
 2. Switch asks controller:
 - "Where should I send this?"
 3. Controller replies with rule
 4. Switch forwards packet
-

Advantages

- Centralized control
 - Easy to configure
 - Programmable network
 - Faster changes
-

Example Use Case

- Data centers
- Cloud providers managing huge networks

• Network Function Virtualization (NFV)

Definition

NFV replaces **hardware network devices** with **software running on servers**

Traditional Approach

- Physical firewall
 - Physical load balancer
 - Physical router
-

NFV Approach

- Firewall → software
 - Load balancer → software
 - Router → software
-

How NFV Works

1. Use virtual machines or containers
2. Run network functions as software
3. Manage them dynamically

Examples of Virtual Network Functions (VNFs)

- Firewall
 - Load balancer
 - Intrusion detection system
-

Advantages

- Cost reduction
- Flexible deployment
- Easy scaling

• Network virtualization

Definition

Creating multiple **virtual networks** on top of a single physical network

Key Idea

- One physical network
- Many isolated virtual networks

How it Works

- Uses:
 - Virtual switches
 - Virtual routers
 - Tunneling protocols

Example

- Cloud provider creates:
 - Separate virtual networks (VPCs) for different users
- All use same physical infrastructure

Types

- Internal virtualization (within data center)
- External virtualization (across networks)

Advantages

- Isolation
- Better resource utilization
- Flexibility

6. How They Work Together (Very Important)

In cloud systems:

- **SDN**
→ Controls traffic
 - **NFV**
→ Provides network services (firewall, LB)
 - **Network Virtualization**
→ Creates isolated networks
-

Combined Flow

User traffic

↓

Virtual Network (Network Virtualization)

↓

Controlled by SDN

↓

Processed by NFV services

UNIT 7: Cloud Resource Management and Scheduling

1. What is Cloud Resource Management?

Managing CPU, memory, storage, and network resources efficiently across multiple virtual machines (VMs)

Goal

- High performance
- Efficient utilization
- Cost optimization

7.1 Resource Provisioning

Definition

Deciding how much resource to allocate to a VM or application

- Static provisioning

Idea

- Resources are **fixed in advance**

Example

- VM created with:
 - 4 CPU
 - 8 GB RAM
- It always uses this configuration

Advantages

- Simple
- Predictable

Disadvantages

- Wastage or shortage possible

- Dynamic provisioning

Idea

- Resources are **adjusted based on demand**

Example

- Traffic increases → add more CPU
 - Traffic decreases → reduce resources
-

Advantages

- Efficient
- Scalable

Disadvantages

- More complex

• Over-provisioning vs Under-provisioning

Over-Provisioning

Meaning

- Allocate **more resources than needed**

Example

- App needs 2 CPU → given 8 CPU

Effect

- Resource waste
 - Higher cost
-

Under-Provisioning

Meaning

- Allocate **less resources than needed**

Example

- App needs 8 CPU → given 2 CPU

Effect

- Poor performance
 - System slowdown
-

One-Line Intuition

- Over → waste
- Under → performance issue

7.2 Resource Allocation

Definition

Deciding which resource goes to which task/VM

- Scheduling techniques

Goal

- Decide:
"Which VM gets CPU time and when?"

Common Techniques

1. **First Come First Serve (FCFS)**
 - Tasks handled in arrival order
2. **Round Robin**
 - Each VM gets equal time slice
3. **Priority Scheduling**
 - Higher priority VM → more resources
4. **Fair Sharing**
 - Equal resource distribution

- Resource allocation policies

Definition

Rules to decide how resources are distributed

Types

1. **Time-Shared Policy**
 - Resources shared over time
2. **Space-Shared Policy**
 - One VM gets full resource at a time

Example

- Time-shared → multiple VMs share CPU
- Space-shared → one VM uses full CPU

7.3 VM Resource Management

- VM placement strategies

Definition

Deciding which physical server will host a VM

Goals

- Load balancing
- Efficient resource usage
- Energy saving

Common Strategies

1. First Fit

- Place VM in first available server

2. Best Fit

- Place VM where it fits best (minimum waste)

3. Load-Based Placement

- Place VM on least loaded server

Example

- Server A (90% used), Server B (30% used)
→ Place VM on Server B

- VM migration techniques

Definition

✦ Get Plus ✕

Moving a VM from one physical machine to another

Types of Migration

1. Live Migration

Idea

- VM runs while moving

Steps

1. Copy memory pages
 2. Sync changes
 3. Switch execution
-

Advantages

- No downtime
-

2. Cold Migration

Idea

- Stop VM first
 - Then move
-

Disadvantages

- Downtime
-

Why Migration is Needed

- Load balancing
- Fault tolerance
- Maintenance



UNIT 8: Cloud Security and Trust Management (Very Important)

8.1 Security Fundamentals

- **CIA Triad:**

1. Confidentiality

Ensuring that **data is accessible only to authorized users**

Techniques

- Encryption
- Access control
- Authentication

Example

- Only logged-in users can view bank details

2. Integrity

Ensuring that **data is not altered or tampered with**

Techniques

- Hashing
- Digital signatures
- Checksums

Example

- Detect if file is modified during transfer

3. Availability

Ensuring that systems and data are accessible when needed

Techniques

- Redundancy
- Load balancing
- Backup systems

Example

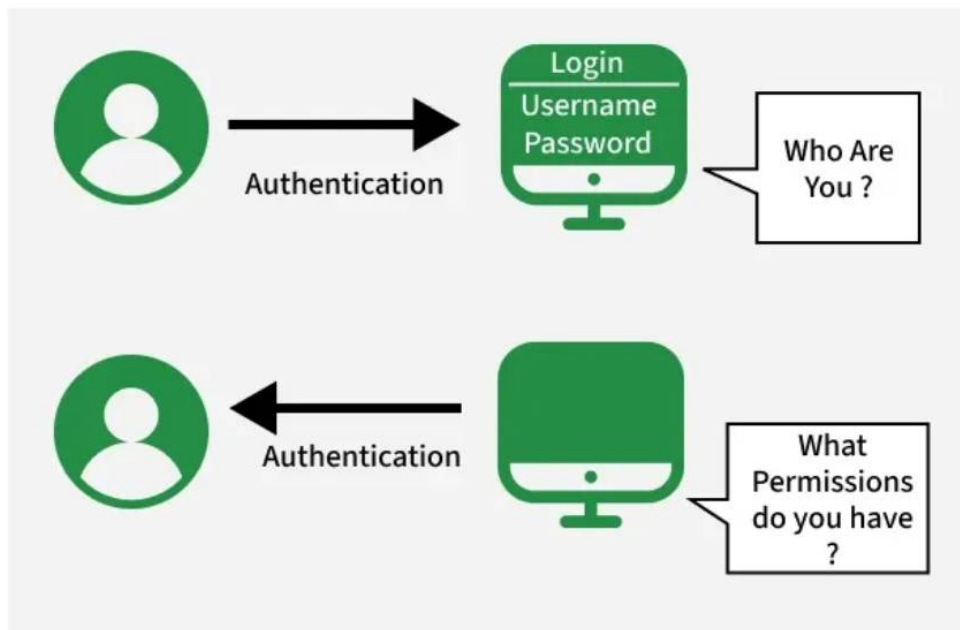
- Website available 24/7

One-Line Intuition

- Confidentiality → "Keep data secret"
- Integrity → "Keep data correct"
- Availability → "Keep system running"

8.2 Identity and Access Management

• Authentication vs Authorization



Feature 	Authentication	Authorization
Core Question	Who are you?	What are you allowed to do?
Purpose	Verifies user identity.	Validates user permissions.
Order	Occurs first.	Occurs after authentication.
Methods	Passwords, MFA, Biometrics	RBAC, ACLs, Permissions
Data Transferred	ID Token	Access Token
Real-world Example	Showing your ID/passport.	Your ticket allowing seat access.
Context	User Login.	User Role/Level (e.g., Admin)

• IAM

Definition

System that controls **user identities and their access permissions**

Functions

- User creation
- Role assignment
- Access control

• Single Sign-On (SSO)

Definition

User logs in **once** and accesses multiple systems

Example

- Login once → access email, cloud dashboard

- **Role-Based Access Control (RBAC)**

Definition

Access is based on **roles**, not individual users

Example

- Admin → full access
- User → limited access

Advantages

- Simplifies management
- Improves security

8.3 Data Security

- **Encryption:**

1. Encryption at Rest

Data stored on disk is encrypted

Example

- Database files encrypted

2. Encryption in Transit

Data encrypted while being transferred

Example

- HTTPS communication

- **Key management**

Definition

Managing encryption keys securely

Includes

- Key generation
- Key storage
- Key rotation

Importance

- If key is compromised → data is exposed

8.4 Threats and Vulnerabilities

1. Data Breaches

- Unauthorized access to sensitive data

2. DDoS Attacks (Distributed Denial of Service)

Definition

Flooding system with traffic to make it unavailable

3. Insider Threats

- Malicious actions by authorized users

4. Multi-Tenancy Risks

Definition

Multiple users share same infrastructure

Risk

- Data leakage between tenants

8.5 Security Models

• Shared responsibility model

Definition

Security responsibility is shared between:

- Cloud provider
- Customer

Responsibilities

Cloud Provider

- Physical security
- Infrastructure
- Network

Customer

- Data
- Applications
- Access control

• Security in IaaS vs SaaS

IaaS

- User controls:
 - OS
 - Applications
- More responsibility

SaaS

- Provider controls:
 - Everything except user data
- Less responsibility

Comparison

Model	User Responsibility
IaaS	High
PaaS	Medium
SaaS	Low

8.6 Compliance and Trust

- **SLAs and security policies**

SLA (Service Level Agreement)

Defines:

- Security guarantees
- Availability
- Responsibilities

Security Policies

- Rules for:
 - Data protection
 - Access control

- **Auditing**

Definition

Monitoring and reviewing system activity

Purpose

- Detect security issues
- Ensure compliance

• Trust models

Definition

Framework defining how trust is established between entities

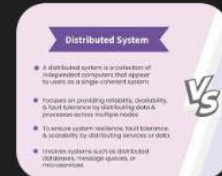
Types (Basic Understanding)

- Trust between:
 - User and provider
 - Services
-

UNIT 9: Cloud Programming and Technologies

9.1 Distributed Computing Concepts

Distributed computing involves multiple independent computers (nodes) connected via a network, working together as a single, cohesive system to achieve shared goals. It offers high scalability, fault tolerance, and efficiency by distributing workloads across resources, allowing systems to remain operational even if individual nodes fail.  GeeksforGeeks +3



Key Components and Characteristics

- **Nodes:** Independent computing devices (servers, workstations) with their own processor and memory.
- **Network:** Interconnects nodes, allowing them to communicate and exchange data.
- **Middleware:** Software that connects different nodes, enabling them to appear as one cohesive system and managing communication/resource sharing.
- **Concurrency:** Multiple processors execute tasks simultaneously.
- **No Shared Clock:** Each node maintains its own local state, necessitating synchronization mechanisms.  Amazon Web Services +5

Common Architectures

- **Client/Server:** Clients request resources from a centralized server.
- **Peer-to-Peer (P2P):** Nodes are equal participants, sharing resources directly with each other.
- **Multi-tier:** A hybrid architecture combining resources to solve complex tasks.  Medium +1

Key Concepts and Benefits

- **Transparency:** The system is hidden from users, who perceive it as a single machine.
- **Fault Tolerance:** The system continues to operate even if parts of it fail.
- **Scalability:** Increased storage or processing power by adding new nodes.
- **Parallel Execution:** Large tasks are broken into subtasks and processed concurrently to reduce time.  GeeksforGeeks +2

Common Applications

- **Cloud Computing:** AWS, Google Cloud, and Azure use this model to provide scalable services.
- **Distributed Databases:** Data is stored across multiple locations for reliability, such as in Banking or Amazon's DynamoDB.
- **Content Delivery Networks (CDNs):** Distributing content closer to users to improve speed, as explained on the [Wikipedia website](#).
- **AI/ML Training:** Large-scale training requiring immense computational power across many nodes, as explained on the [IBM website](#).  Wikipedia +4

9.2 Web Services

Web services are software systems designed to support machine-to-machine interaction over a network. They are the most common and preferred way to implement SOA, leveraging a set of standards to provide a common approach for defining, publishing, and using services. The two main styles for web services are SOAP and REST.  Oracle +3

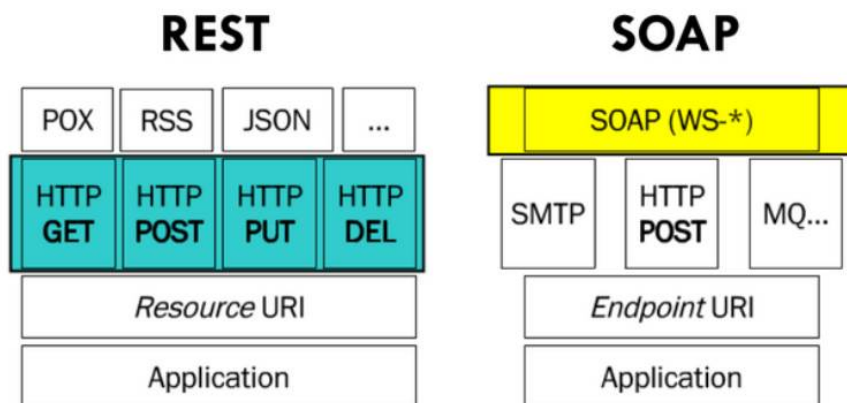
• Service-Oriented Architecture (SOA)

SOA is a design approach that structures an application as a collection of loosely coupled, interoperable services. The primary goals are to promote reusability, flexibility, and the ability to integrate diverse applications across different platforms and languages. 🚫 Oracle +3

• Key Principles:

- **Loose Coupling:** Services should have minimal dependencies on external resources, allowing for independent updates without impacting other components.
- **Interoperability:** Services can work together regardless of their underlying platform or programming language.
- **Abstraction:** The implementation logic of a service is hidden from consumers, who only interact with its defined interface.
- **Reusability:** Services are designed as self-contained business functions that can be reused across different applications within an enterprise. 🚫 Oracle +2

• REST APIs vs SOAP



	REST	SOAP
Definition	REST isn't a protocol but an architectural style introduced by Roy Fielding in his doctoral dissertation 2000. It treats objects on a server as resources that can be accessed with a standard set of methods. These resources are addressed using URLs, and the communication is typically over HTTP.	Originally developed by Microsoft in the late 1990s, SOAP is a protocol for exchanging structured information in web services using XML. It can be implemented in various programming languages and on multiple platforms. SOAP is protocol-agnostic, meaning it

	REST	SOAP
		can work with any protocol, but it's most commonly used with HTTP and SMTP.
Message Format	Typically uses JSON for its message format, which tends to be more lightweight and more accessible to parse than XML. However, REST can also use XML, HTML, or any other format.	Uses XML exclusively for its message format. The messages are standardized and often contain much information, including headers and bodies.
State	Stateless by design. All the required information should be available within the individual request. This ensures that each request from a client contains all the info the server needs to perform the request.	Generally considered stateful, as it can maintain state through sessions.
Performance	Typically faster and more performant, primarily when using JSON. The lightweight message format reduces the amount of data to be transferred and processed.	Tends to be slower due to its extensive XML format, which requires parsing. Additionally, the protocol often contains much additional information in the message that might only sometimes be necessary.
Methods	Uses standard CRUD operations (Create, Read, Update, Delete) mapped to HTTP methods like POST, GET, PUT, and DELETE.	Has a set of standard operations, but developers can define arbitrary operations in the service.
Error Handling	Typically uses standard HTTP status codes, like 404 for "Not Found" or 500 for "Internal Server Error."	Has built-in error handling through standardized fault elements.
Security	Relies on underlying protocols, like HTTPS, for security. Authentication can be achieved using OAuth, JSON Web Tokens (JWT), and other mechanisms.	Offers built-in security through the WS-Security standard. This provides a suite of tools, including token propagation, message integrity, and message confidentiality.

REST		SOAP	
Pros	Cons	Pros	Cons
Lightweight	Point-to-point communication	Easy to consume(usually)	More difficult to setup
Human Readable	Lack of standards	Standards (WSDL, etc)	More verbose
Easier to build	Tied to HTTP	Distributed computing	Harder to develop

9.3 Application Architecture

• Microservices architecture

Microservices architecture is **an approach to software development that structures an application as a collection of small, independent, and loosely coupled services designed around specific business capabilities**. Each service runs in its own process and communicates with others using lightweight mechanisms, typically APIs, enabling independent development, deployment, and scaling.  Microsoft Learn +3

Key Characteristics

- **Independently Deployable:** Each service can be built, tested, and deployed without affecting other services, enabling faster, more frequent updates.
- **Organized Around Business Capabilities:** Services are designed to support specific business functions (e.g., inventory management, payment processing), rather than technical layers like "UI" or "database".
- **Small, Focused Teams:** Small, autonomous, cross-functional teams can own a service's entire lifecycle, from development to operations, which promotes agility and clear ownership.
- **Technology Diversity (Polyglot):** Teams have the flexibility to choose the best technology stack, programming language, and database for their specific service's needs.
- **Data Autonomy:** Each microservice manages its own dedicated database to maintain data isolation and ensure loose coupling.  Microsoft Learn +3

Core Components

- **API Gateway:** The single entry point for all client requests, responsible for routing, authentication, and load balancing across various services.
- **Service Registry & Discovery:** A mechanism that keeps track of the location and status of all available service instances, allowing them to find each other dynamically.
- **Inter-service Communication:** Services communicate using lightweight protocols such as RESTful HTTP APIs or asynchronous messaging systems/event brokers like Apache Kafka or RabbitMQ.
- **Containerization & Orchestration:** Technologies like Docker are used to package services and their dependencies into containers, while Kubernetes automates the deployment, scaling, and management of these containers at scale.
- **Observability:** Essential for monitoring the health of a distributed system, involving centralized logging, distributed tracing, and real-time performance metrics.  Microsoft Learn +5

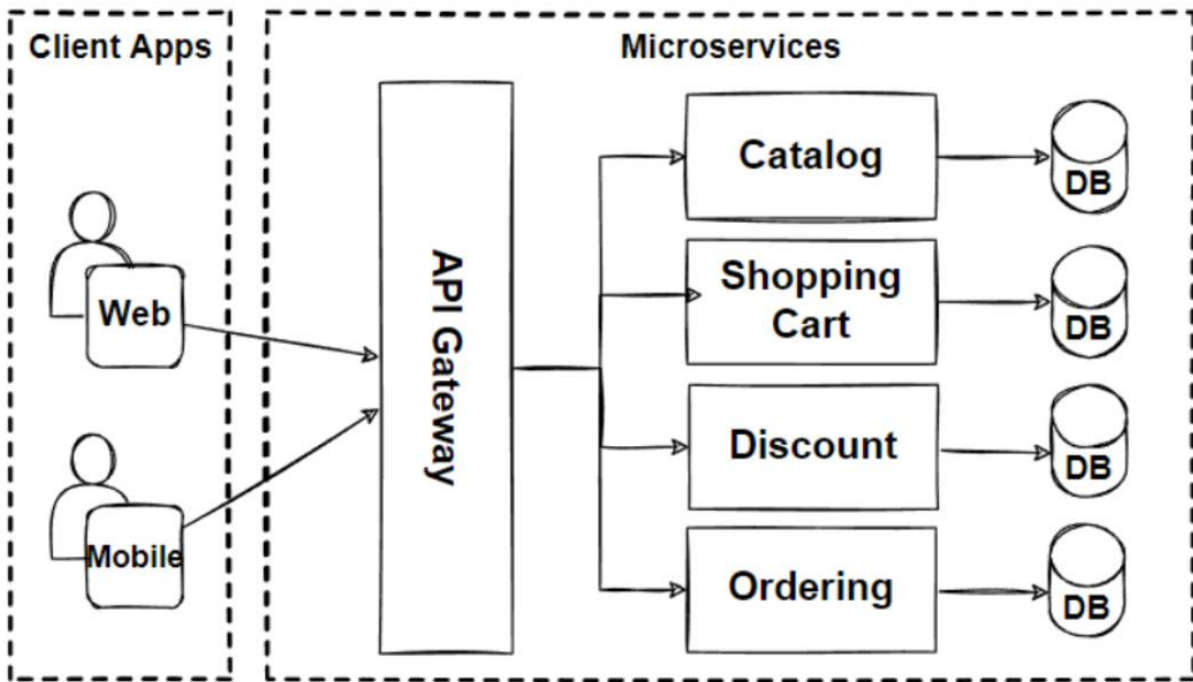
Advantages and Disadvantages

Advantages

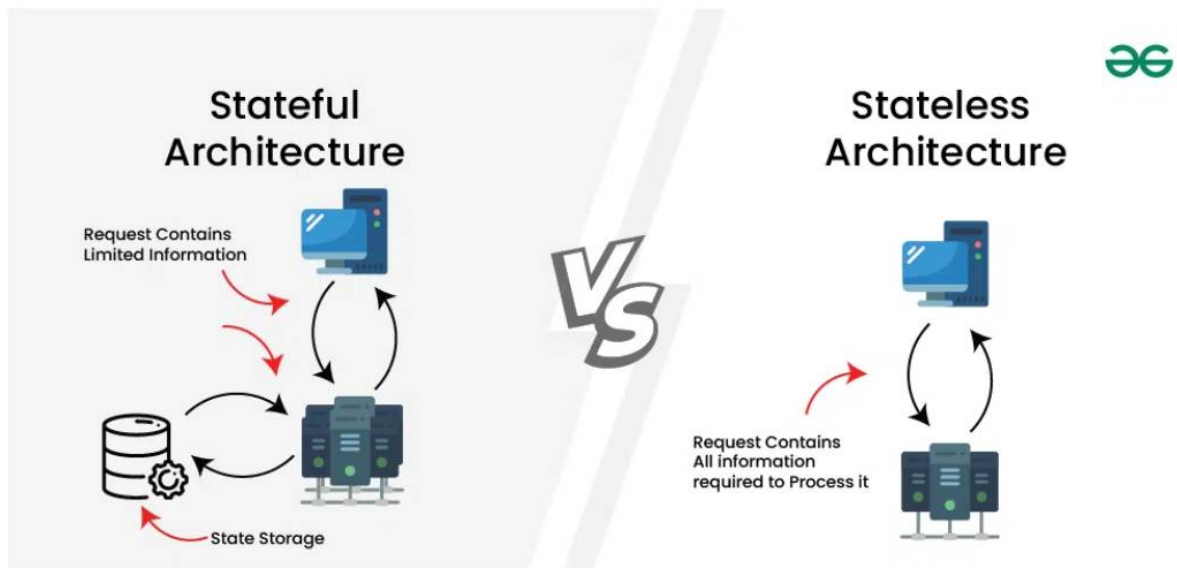
- * **Improved Scalability:** Individual services can be scaled independently based on demand, optimizing resource usage.
- * **Enhanced Agility & Faster Time-to-Market:** Independent deployment enables faster feature releases and bug fixes without affecting the whole application.
- * **Better Fault Isolation:** If one service fails, the entire application isn't necessarily disrupted, improving overall system resilience.
- * **Technological Flexibility:** Allows teams to select the best language or framework for each specific job.

Disadvantages

- * **Increased Complexity:** The entire system has more moving parts and introduces challenges like network latency and data consistency management.
- * **Testing and Debugging Challenges:** Testing service dependencies and debugging issues across multiple services can be more difficult than in a monolithic application.
- * **Operational Overhead:** Requires a mature DevOps culture and significant investment in automation for deployment, monitoring, and management.
- * **Data Integrity & Consistency:** Managing data consistency across multiple, independent databases can be complex and often requires implementing specific architectural patterns like the Saga pattern.



- **Stateful vs Stateless systems**



Stateful Architecture	Stateless Architecture
Scaling requires synchronization of session data.	Horizontal scaling is straightforward.
Failure in one server can affect sessions stored on it.	Failures are isolated, impacting only individual requests.
May experience increased latency due to session management.	Typically faster response times due to lack of session overhead.
Requires more resources to store and manage session state.	Uses resources efficiently because no session state is stored.
Caching can be complex due to session-specific data.	Caching is simpler since requests are independent.
Deployment can be complex because session data must be synchronized.	Deployment and maintenance are easier due to stateless nature.
Maintains session context to ensure transaction continuity.	Transactions are handled independently at the request level.
Load balancing may require session affinity (sticky sessions).	Load balancing is simpler since any server can handle any request.
Developers must manage session handling and related issues.	Developers can focus mainly on business logic without session concerns.

Benefits of Stateful Architecture

Stateful architecture provides several advantages when applications need to maintain user sessions and context across multiple requests.

- **Session Persistence:** Maintains user sessions, allowing smooth transitions across steps or devices.
- **Efficient Resource Use:** Stores session data on the server, reducing repeated transfers and processing.
- **Personalization:** Uses past interactions to deliver tailored experiences, like recommendations.
- **Enhanced Security:** Centralized session management supports strong authentication and encryption.

Benefits of Stateless Architecture

Stateless architecture offers advantages in scalability and simplicity because each request is handled independently.

- **High Scalability:** Easily handles large numbers of requests without session management.
- **Fault Tolerance:** Each request is independent, so failures in one area don't affect others.
- **Simplified Load Balancing:** Requests can be evenly distributed without sticky sessions.
- **Better Performance:** No session overhead, resulting in faster responses and lower latency.

9.4 APIs

• REST principles

To be considered "RESTful," an API must adhere to several architectural constraints:  IBM +1

- **Client-Server Architecture:** The client (e.g., a web browser or mobile app) and the server must be independent of each other. This separation allows each component to evolve independently.
- **Statelessness:** Each request from the client to the server must contain all the information necessary to process it. The server cannot store any client context or session data between requests, which improves scalability and reliability.
- **Cacheability:** Responses should be able to be cached on the client or an intermediary to improve performance and reduce server load. The server must provide information in its responses about whether caching is permitted.
- **Uniform Interface:** This is a core principle for decoupling client and server implementations. It involves a standardized way of interacting with resources, including using standard HTTP methods and consistent URI patterns.
- **Layered System:** The architecture can involve multiple layers (e.g., security, load balancers) without the client being aware of them, which aids in scalability and security.
- **Code on Demand (Optional):** Servers can optionally send executable code (like Java applets) to extend client functionality.  Amazon Web Services +6

• HTTP methods

REST APIs use standard HTTP methods to perform operations on resources, mapping directly to common CRUD (Create, Read, Update, Delete) operations:  [GeeksforGeeks +1](#)

- **GET** : Retrieves a representation of a resource or collection of resources. **GET** requests are considered safe (do not modify server state) and idempotent (multiple identical requests have the same effect as a single one).
 - *Example:* **GET** /users/123
 - **POST** : Submits data to the server to create a new resource, typically within a collection. **POST** is neither safe nor idempotent; multiple identical requests will result in multiple distinct resources being created.
 - *Example:* **POST** /users (with user data in the request body)
 - **PUT** : Updates an existing resource with the complete new data provided in the request body. If the resource does not exist, it may create a new one, depending on the API design. **PUT** is idempotent.
 - *Example:* **PUT** /users/123 (with complete user data in the request body)
 - **PATCH** : Applies partial modifications to a resource. Unlike **PUT**, it only sends the fields to be modified, not the entire resource representation. **PATCH** is not inherently idempotent.
 - *Example:* **PATCH** /users/123 (with only the fields to be updated in the request body)
 - **DELETE** : Removes a specific resource identified by its URI. **DELETE** is considered an idempotent method.
 - *Example:* **DELETE** /users/123  [Postman Blog +5](#)
-

UNIT 10: DevOps and Cloud Integration

10.1 DevOps Basics

What is DevOps?

DevOps = **Development + Operations**, a methodology that integrates software development and IT operations to enable **faster and reliable software delivery**

Key Goals

- Faster deployment
- Continuous delivery
- Automation
- Collaboration between teams

Traditional vs DevOps

Traditional	DevOps
Separate teams	Integrated teams
Slow releases	Frequent releases
Manual processes	Automated pipelines

• CI/CD pipeline

CI/CD Pipeline (Very Important)

CI/CD = Continuous Integration + Continuous Delivery/Deployment

Pipeline Stages

1. Code
2. Build
3. Test
4. Deploy
5. Monitor

Continuous Integration (CI)

Definition

Developers frequently **merge code into a shared repository**, and it is automatically tested

Key Points

- Detect bugs early
- Automated testing
- Frequent commits

Continuous Deployment (CD)

Definition

Code changes are **automatically deployed to production** after testing

Difference

Concept	Meaning
Continuous Delivery	Ready to deploy (manual trigger)
Continuous Deployment	Automatically deployed

10.2 Infrastructure as Code (IaC)

Definition

Managing and provisioning infrastructure using **code instead of manual setup**

How it works

- Write configuration files
- Automatically create:
 - VMs
 - Networks
 - Storage

Example

- Instead of manually creating server → write script to create it

Benefits

- Automation
- Consistency
- Version control
- Faster deployment

Types of IaC

1. Declarative

- Define **what** you want

2. Imperative

- Define **how** to achieve it

10.3 Monitoring and Logging

• Logging systems

Definition

Recording events and activities in the system

What is logged

- Errors
- Requests
- System events

Purpose

- Debugging
- Auditing
- Troubleshooting

• Monitoring tools

Definition

Continuously tracking system performance and health

Metrics Monitored

- CPU usage
- Memory usage
- Network traffic
- Response time

10.4 Tools (Conceptual)

• Jenkins

- **Automation Server:** Jenkins runs as a standalone application written in Java, providing a web-based interface to manage automation tasks.
- **Plugins:** Jenkins has a vast ecosystem of over a thousand plugins that allow it to integrate with virtually any tool or service, such as Git, Maven, Docker, and Kubernetes.
- **Jobs (Projects):** These are the basic units of work in Jenkins, defining specific tasks or processes to be performed, such as compiling code, running tests, or deploying an application.
- **Pipeline:** A series of automated steps that define the entire CI/CD workflow as code, using a text file called a `Jenkinsfile`.
 - **Declarative Pipeline:** A simpler, more structured syntax recommended for most use cases, emphasizing readability.
 - **Scripted Pipeline:** A more flexible syntax based on Groovy scripting, offering greater control for complex workflows.
- **Controller and Agents:** Jenkins uses a distributed architecture where a central controller manages multiple agents (formerly master/slave) to distribute workloads and run tasks in parallel, improving performance and scalability.  GeeksforGeeks +9

How Jenkins Works

The typical workflow in a Jenkins environment is as follows:  Hashnode

1. **Code Commit:** A developer pushes code changes to a source code repository (e.g., Git, GitHub).
2. **Detection:** Jenkins automatically detects the changes (often via webhooks or polling the SCM periodically).
3. **Build:** Jenkins triggers a build job that compiles the source code, resolves dependencies, and packages the application (e.g., using Maven or Gradle).
4. **Test:** Automated tests (unit, integration, etc.) are executed to verify the code quality and functionality.
5. **Deployment:** If all tests pass, the application is automatically deployed to a staging or production environment.
6. **Notification:** Stakeholders are notified of the build status (success or failure). 

Core Benefits

- **Automation:** Automates repetitive tasks, reducing manual effort and human error.
- **Early Feedback:** Catches bugs and integration issues early in the development cycle, making them easier and faster to fix.
- **Speed:** Accelerates the software delivery process from development to production.
- **Extensibility:** Its vast plugin ecosystem makes it adaptable to almost any technology stack and environment.  GeeksforGeeks +3

• Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. It groups containers into logical units for easy management across a cluster of machines.  Kubernetes +3

Key Concepts

- **Cluster:** A set of machines, called nodes, that work together as a single unit.
- **Control Plane (Master Node):** Manages the worker nodes and the pods in the cluster, coordinating all activities. Key components include the API Server (front end for the control plane), etcd (distributed key-value store for cluster state), Scheduler (assigns pods to nodes), and Controller Manager (ensures the cluster's current state matches the desired state).
- **Node (Worker Node):** The machines (physical or virtual) that run the containerized applications. Each node runs a container runtime (like containerd), a `kubelet` (an agent that communicates with the control plane), and `kube-proxy` (a network proxy).
- **Pod:** The smallest deployable unit in Kubernetes, representing a single instance of a running process. A pod encapsulates one or more containers, shared storage, and an IP address.
- **Deployment:** A higher-level object that manages a set of identical pods, ensuring that a specified number of replicas are running and providing declarative updates and rollbacks for applications.
- **Service:** An abstraction that defines a logical set of pods and provides a stable network endpoint (IP address and DNS name) to access them, handling load balancing and service discovery.
- **kubectl:** The command-line interface (CLI) tool used to interact with the Kubernetes API server and manage the cluster.  Okteto +7

Core Features

- **Automated Rollouts and Rollbacks:** Kubernetes manages the rollout of application updates while monitoring health and can automatically roll back changes if something goes wrong.
- **Self-Healing:** It automatically restarts failed containers, replaces unhealthy pods, and reschedules them onto healthy nodes to maintain the desired state.
- **Horizontal Scaling:** Applications can be scaled in or out (up or down) automatically based on resource usage, such as CPU usage, or with a simple command.
- **Secret and Configuration Management:** Kubernetes allows you to manage sensitive information (secrets) and general configuration data separately from the application code, without rebuilding container images.
- **Storage Orchestration:** It automatically mounts the storage system of your choice, whether local storage, a public cloud provider, or a network storage system.  Kubernetes +1

For hands-on learning, the official [Kubernetes documentation](#) offers a beginner's tutorial using Minikube, which sets up a local Kubernetes cluster on your machine.  Kubernetes +2

UNIT 11: Advanced and Emerging Topics

11.1 Serverless Computing

- **FaaS (Function as a Service)**

Serverless Function as a Service (FaaS) is a cloud computing model allowing developers to run code snippets (functions) in response to events without managing servers. FaaS automatically scales, provides high availability, and charges only for execution time—not idle capacity—offering a cost-effective, event-driven architecture for APIs, IoT, and backend tasks.  GeeksforGeeks +4

Key Aspects of Serverless FaaS:

- **No Server Management:** Cloud providers (e.g., [AWS Lambda](#), [Google Cloud Functions](#), Azure Functions) manage all infrastructure, including OS maintenance, capacity provisioning, and server scaling.
- **Event-Driven:** Functions are executed in direct response to events like HTTP requests (via API Gateway), file uploads, or database changes.
- **Automatic Scaling:** The infrastructure scales automatically from zero to thousands of concurrent requests based on demand, ensuring high availability.
- **Pay-As-You-Go Billing:** Charges are based strictly on the number of requests and the duration (often in milliseconds) of execution, eliminating costs for idle resources.
- **Stateless:** Functions are designed to be short-lived and ephemeral, storing state in external databases or storage services.  GeeksforGeeks +6

Benefits:

- **Focus on Code:** Developers concentrate solely on business logic rather than infrastructure.
- **Cost Efficiency:** Significant savings are possible, as you only pay when code executes.
- **Rapid Deployment:** Code can be deployed instantly, speeding up development cycles.  GeeksforGeeks +3

Use Cases:

- **Web Backends:** Handling API requests for mobile or web applications.
- **Data Processing:** Real-time processing of files (e.g., image resizing) or data streams.
- **IoT & Automation:** Processing messages from IoT devices and managing automated workflows.  IBM +4

Challenges:

- **Cold Starts:** Initial latency when a function is triggered after being idle.
- **Vendor Lock-in:** Moving code between providers can be difficult.
- **Debugging:** Complex monitoring due to distributed, ephemeral nature.  PayPro Global +3

AWS Serverless Platform

AWS provides a fully managed serverless platform to deploy and run your application. Let's look into the services provided by AWS serverless platform.

Compute: AWS provides services such as [AWS Lambda](#) to run the code without provisioning or managing servers, [Lambda@Edge](#) to respond to events generated by Amazon CloudFront events, and [AWS Fargate](#) facilitates a serverless compute engine for containers.

Storage: AWS provides [Amazon Simple Storage Service](#) (Amazon S3) that ensures secure, durable, highly-scalable **object storage**, and [Amazon Elastic File System](#) (Amazon EFS) provides simple, scalable, elastic **file storage**.

Data Stores: [Amazon DynamoDB](#) provides NoSQL database service, and [Amazon Aurora Serverless](#) (for [Amazon Aurora](#)) permits automatic database start-up, shut down, and scale capacity up or down. AWS also offers database proxy services using [Amazon RDS Proxy](#).

API Proxy: [Amazon API Gateway](#) service helps developers to create, publish, maintain, and secure APIs at any scale. Here, it allows you to process hundreds of thousands of concurrent API calls by efficiently handling traffic.

Application Integration: It facilitates services such as [Amazon SNS](#) (pub or sub messaging service), [Amazon SQS](#) (message queuing service), [AWS AppSync](#) (to securely access, manipulate, and combine data), and [Amazon EventBridge](#) (to transfer application data from different sources to your AWS environment).

Orchestration: [AWS Step Functions](#) orchestrates the components of distributed applications and microservices.

Analytics: AWS provides services such as [Amazon Kinesis](#) (for streaming data on AWS) and [Amazon Athena](#) (an interactive query service to analyze data) to deal with data analytics.

Apart from the above services, it also provides a wide array of [tools](#) for the serverless application development process. These tools include continuous integration and testing tools, deployment, monitoring, and diagnostics tools, frameworks, and IDE plugins.

11.2 Edge and Fog Computing

Fog Computing is the term introduced by Cisco that refers to extending [cloud computing](#) to an edge of the enterprise's network. Thus, it is also known as [Edge Computing](#) or Fogging. It facilitates the operation of computing, storage, and networking services between end devices and computing data centers.

Types of Fog Computing

- **Device-level Fog Computing:** Device-level fog computing utilizes low-power technology, including sensors, switches, and routers. It can be used to collect data from these devices and upload it to the cloud for analysis.
- **Edge-level Fog Computing:** Edge-level fog computing utilizes network-connected servers or appliances. These devices can be used to process data before it is uploaded to the cloud.

- **Gateway-level Fog Computing:** Fog computing at the gateway level uses devices to connect the edge to the cloud. These devices can be used to control traffic and send only relevant data to the cloud.
- **Cloud-level Fog Computing:** Cloud-level fog computing uses cloud-based servers or appliances. These devices can be used to process data before it is sent to end users.

Components of Fog Computing

- **Edge devices:** Edge devices are the network devices nearest to the data source. Edge devices consist of sensors, [PLCs \(programmable logic controllers\)](#), and gateway routers.
- **Data Processing:** Data processing occurs locally on edge devices rather than being routed to a central location for processing. The end effect is greater performance and lower [latency](#).
- **Data Storage:** in Data storage. Instead of transferring data to a central place, edge devices can keep information locally. This increases security and privacy while lowering latency.
- **Connectivity:** For fog computing to work, edge devices must be connected to the rest of the network at high speeds. This can be done using wired or wireless methods.

When to Use Fog Computing?

- It is used when only selected data is required to send to the cloud. This selected data is chosen for long-term storage and is less frequently accessed by the host.

- It is used when the data should be analyzed within a fraction of seconds i.e Latency should be low.
- It is used whenever a large number of services need to be provided over a large area at different geographical locations.
- Devices that are subjected to rigorous computations and processings must use fog computing.
- Real-world examples where fog computing is used are in IoT devices Devices with Sensors, Cameras (IIoT-Industrial [Internet of Things](#)), etc.

Advantages of Fog Computing

- This approach reduces the amount of data that needs to be sent to the cloud.
- Since the distance to be traveled by the data is reduced, it results in saving [network bandwidth](#).
- Reduces the response time of the system.
- It improves the overall security of the system as the data resides close to the host.
- It provides better privacy as industries can perform analysis on their data locally.

Disadvantages of Fog Computing

- Congestion may occur between the host and the fog node due to increased traffic (heavy data flow).
- Power consumption increases when another layer is placed between the host and the cloud.

- Scheduling tasks between host and fog nodes along with fog nodes and the cloud is difficult.
- Data management becomes tedious as along with the data stored and computed, the transmission of data involves encryption-decryption too which in turn release data.

Applications of Fog Computing

- It can be used to monitor and analyze the patients' condition. In case of emergency, doctors can be alerted.
- It can be used for real-time rail monitoring as for high-speed trains we want as little latency as possible.
- It can be used for gas and oils pipeline optimization. It generates a huge amount of data and it is inefficient to store all data into the cloud for analysis.

11.3 Cloud Simulators

• CloudSim basics

CloudSim is a free, open-source Java-based framework used for modeling, simulating, and testing cloud computing environments, allowing developers to test resource provisioning and scheduling algorithms without high-spec hardware. It manages data centers, virtual machines (VMs), and resource allocation via user-defined policies to analyze performance bottlenecks.  GeeksforGeeks +2

What are the Basic Components and Features of CloudSim?

Key Basics of CloudSim:

- **Architecture Layers:** Consists of user code (scenarios), CloudSim layer (VMs, hosts, cloudlets), and the simulation engine (resource management).
- **Key Components:**
 - **Datacenter:** Represents a physical infrastructure unit (server).
 - **Host:** Models a physical computing node with RAM, bandwidth, and CPU.
 - **VM (Virtual Machine):** Virtualized instances created on hosts.
 - **Cloudlet:** Models the application tasks or service requests (e.g., CPU load).
 - **Broker:** Manages scheduling and mapping of cloudlets to VMs and VMs to hosts.

- **Core Capabilities:**
 - Modeling large-scale data centers.
 - Simulating virtualized server hosts with customized policies.
 - Simulating energy-aware resources and network topologies.
- **Setup:** Requires JDK and an IDE like Eclipse or NetBeans; it does not require installation, just adding libraries to the classpath.  GeeksforGeeks +5

Common Simulation Flow:

1. Initialize the CloudSim package.
2. Create DatacenterBroker.
3. Create Datacenters, Hosts, and VMs.
4. Create Cloudlets (tasks) and submit them to the broker.
5. Start the simulation.
6. Analyze the output results.  GeeksforGeeks +3

11.4 Distributed System Algorithms

• Consensus algorithms:

1. What is Consensus in Distributed Systems?

Consensus =

| All nodes agree on the same value/state even if failures occur

Why needed?

- Multiple machines (nodes)
- Network failures possible
- Need **one consistent decision**

Examples

- Leader election
- Committing a transaction
- Replicating logs

2. Core Problem (Very Important Intuition)

Imagine:

- 5 servers
- Need to agree on a value (say: "commit transaction")

Problems:

- Messages can be delayed
- Some nodes may crash
- Multiple nodes may propose different values

Goal:

| "Everyone agrees on ONE value"

3. Basic Idea Behind Consensus

To decide a value:

- Nodes **propose**
- Nodes **vote**
- Majority decides

Key rule:

| Majority (quorum) agreement = final decision

○ Paxos

4. Paxos Algorithm (Classic but Hard)

Key Idea

Agreement happens in **two phases** using **proposers and acceptors**

Roles in Paxos

- **Proposer** → suggests value
- **Acceptor** → votes
- **Learner** → learns final result

Working of Paxos (Simplified)

Phase 1: Prepare Phase

1. Proposer sends:
 - “Prepare request with proposal number N”
2. Acceptors reply:
 - “I promise not to accept smaller proposals”
 - Send previously accepted value (if any)

Phase 2: Accept Phase

3. Proposer chooses value:
 - Either new value OR previously accepted value
4. Sends:
 - “Accept request (N, value)”
5. Acceptors accept if:
 - They didn't promise a higher proposal

Final Decision

- If **majority accepts**, value is chosen

Key Concepts in Paxos

- Proposal number ensures ordering
 - Majority ensures consistency
 - Even if nodes fail → system still works
-

Problem with Paxos

- Hard to understand
- Hard to implement

○ Raft

5. Raft Algorithm (Exam Favorite – Easy Version of Paxos)

Key Idea

Simplifies consensus using a **leader-based approach**

Roles in Raft

- **Leader** → controls everything
- **Followers** → obey leader
- **Candidate** → tries to become leader

Working of Raft

Step 1: Leader Election

- If no leader:
 - Nodes become **candidates**
 - Ask for votes
 - Majority vote → becomes leader
-

Step 2: Log Replication

1. Client sends request to leader
 2. Leader appends request to its log
 3. Leader sends log to followers
 4. Followers acknowledge
-

Step 3: Commit

- Once **majority confirms**, entry is committed
- All nodes apply the same change

Failure Handling

If Leader Fails:

- Followers detect timeout
- New election starts

6. Paxos vs Raft (Very Important)

Feature	Paxos	Raft	
Approach	Multi-proposer	Leader-based	
Complexity	Hard	Easy	
Readability	Low	High	
Implementation	Difficult	Practical	
Exam usage	Conceptual	Frequently asked	

7. Key Technical Concepts (Must Know)

Quorum (Majority)

- More than half nodes
- Ensures no conflicting decisions

Safety Property

- Only one value is chosen

Liveness Property

- System eventually makes progress

Log Replication

- Same sequence of operations on all nodes

8. One-Line Intuition

- Paxos:
 - | "Careful voting system with strict rules"
- Raft:
 - | "One leader decides, others follow"

UNIT 12: Cloud Economics and SLA

12.1 Cost Models

• Pay-as-you-go

- **Definition:** A consumption-based model charging only for resources actually used, such as compute hours or storage capacity.
- **Pros:** High flexibility, no long-term contracts, ideal for unpredictable, fluctuating, or bursty workloads.
- **Cons:** Higher per-unit costs compared to reserved instances; potentially volatile bills if usage isn't managed.
- **Best Use Case:** New projects, developers, and startups with uncertain capacity needs.  DigitalOcean +5

• Subscription models

- **Definition:** Recurring, fixed payments (monthly/yearly) for a predetermined bundle of cloud resources.
- **Pros:** Predictable spending for easy budgeting and often cheaper for high, steady-state usage.
- **Cons:** Risk of paying for idle resources if usage is overprovisioned, and lower flexibility.
- **Best Use Case:** Long-term projects with stable, predictable resource consumption.  CloudKeeper +4

12.2 Billing

Cloud providers use varied strategies to balance operational expenditure (OpEx) with performance requirements: 

- **Pay-as-You-Go (On-Demand):** The standard model where customers pay for resources hourly or per-second (e.g., compute hours, GB stored). It offers high flexibility but carries the highest per-unit cost.
- **Reserved Instances/Committed Use Discounts:** Customers commit to a minimum usage level (1–3 years) in exchange for deep discounts, typically 30%–70% lower than on-demand rates.
- **Spot Instances/Preemptible VMs:** Providers sell unused capacity at discounts up to 90%. Ideal for fault-tolerant, non-critical workloads, but instances can be terminated with little notice.
- **Tiered Pricing:** Usage-based tiers (e.g., free tier for first X units, lower prices for higher volume).
- **Hybrid Billing:** Combining subscription fees with usage-based charges for enterprise customers needing custom SLAs and high flexibility.  Cloud Assert +4

12.3 SLA (Service Level Agreements) Parameters

SLAs define the legally binding performance levels. If not met, providers issue financial credits (e.g., 10%–100% of the monthly bill).  Google Cloud +1

• Availability (e.g., 99.9%, 99.99%)

1. Availability (Uptime)

Availability measures the percentage of time a service is accessible. 

- **99.9% ("Three Nines"):** Approximately 43 minutes of allowed downtime per month. Typical for business-critical applications.
- **99.99% ("Four Nines"):** Approximately 4 minutes of allowed downtime per month. Required for highly critical infrastructure (e.g., payments, auth).
- **99.999% ("Five Nines"):** ~26 seconds of downtime a month.
- *Note:* Cloud providers usually require multi-zone or redundant architectures to achieve 99.99% or higher.  UptimeRobot +5

• Throughput

Throughput measures the amount of work completed in a given period (e.g., requests per second, data transfer volume).  Medium +1

- **SLA Impact:** Higher throughput requirements necessitate greater resource allocation, increasing costs.
- **Network Tiers:** Premium Tier networks provide higher throughput and lower latency compared to Standard Tier, which relies on public internet.  Google Cloud +2

• Latency

Latency measures the time delay between a request and a response, usually in milliseconds (ms).  Amazon Web Services

- **SLA Impact:** Performance-sensitive applications (e.g., real-time analytics, gaming) often require specific latency targets, such as P95/P99 latency (the latency at which 95% or 99% of requests are faster than a certain time).
 - **Geographical Factor:** Lower latency is often achieved by choosing regions closer to users or utilizing specialized networking.  Google Cloud +4
-

UNIT 13: Cloud Platforms (Conceptual Overview)

13.1 Major Providers

- **AWS:**

Amazon Web Services (AWS) is a comprehensive cloud platform that includes services such as EC2 (Elastic Compute Cloud) for virtual servers, S3 (Simple Storage Service) for object storage, and IAM (Identity and Access Management) for securely managing access to resources.  Amazon.com +4

- **EC2**

Amazon EC2 (Elastic Compute Cloud)

- **Function:** EC2 provides resizable compute capacity in the cloud as virtual servers, known as instances.
- **Use Cases:** It is used for running application hosts, performing batch processing tasks, and hosting web servers.
- **Key Feature:** Users can select Amazon Machine Images (AMIs) to launch instances with specific operating systems and software configurations.  Amazon AWS Documentation +4

- **S3**

Amazon S3 (Simple Storage Service)

- **Function:** S3 is an object storage service designed for high durability (99.999999999%) and availability.
- **Use Cases:** It is used for data archiving, backup and restoration, hosting static websites, and acting as a data lake for analytics.
- **Key Feature:** Data is stored in "buckets" and accessed via a simple web services interface.  Amazon AWS Documentation +4

◦ IAM

AWS IAM (Identity and Access Management)

- **Function:** IAM is a security framework that controls who can be authenticated and authorized to use AWS resources.
- **Use Cases:** It manages users, groups, and roles, and assigns permissions using policies, ensuring adherence to the principle of least privilege.
- **Key Feature:** IAM roles are crucial for securely granting permissions to AWS services (like an EC2 instance accessing an S3 bucket) without embedding credentials directly in code. Mk Rajat Arora - Medium +4

• AWS vs Microsoft Azure vs Google Cloud Platform

Feature ⓘ	AWS	Azure	Google Cloud
Best For	Scalability, broad service catalog, global reach	Microsoft users, hybrid cloud environments	AI, data analytics, open-source technologies
Market Share (Q2 2025)	30% (Market Leader)	20% (Second largest)	13% (Third largest)
Service Count	Largest catalog (over 200)	Extensive range (over 200)	Comprehensive range (over 200)
Pricing Model	Pay-as-you-go, Reserved Instances	Pay-as-you-go, Hybrid Benefit	Per-second billing, Sustained Use Discounts
Learning Curve	Steeper due to vastness of services	Easier for those familiar with Microsoft tools	User-friendly interface, easier to navigate



Virtual Servers	Instances	VMs	VM Instances
Platform-as-a-Service	Elastic Beanstalk	Cloud Services	App Engine
Serverless Computing	Lambda	Azure Functions	Cloud Functions
Docker Management	ECS	Container Service	Container Engine
Kubernetes Management	EKS	Kubernetes Service	Kubernetes Engine
Object Storage	S3	Block Blob	Cloud Storage
Archive Storage	Glacier	Archive Storage	Coldline
File Storage	EFS	Azure Files	ZFS / Avere
Global Content Delivery	CloudFront	Delivery Network	Cloud CDN
Managed Data Warehouse	Redshift	SQL Warehouse	Big Query

Key Strengths and Use Cases

- **Amazon Web Services (AWS):** Ideal for enterprises and startups requiring maximum flexibility, reliability, and the most comprehensive range of services and third-party integrations. Its mature ecosystem is the industry benchmark for cloud services quality. You can explore their offerings on the Amazon Web Services (AWS) site.
- **Microsoft Azure:** The natural fit for organizations already using Microsoft products like Windows Server, Office 365, and Active Directory. It offers seamless integration and cost benefits via the Azure Hybrid Benefit, making it a top choice for enterprise and hybrid cloud strategies. More information can be found on the Microsoft Azure site.
- **Google Cloud Platform (GCP):** A strong contender for companies focused on big data, AI/ML, and open-source technologies. It is known for its high-performance networking and developer-friendly platform, making it a popular choice for startups and tech-savvy companies. You can learn more about their services on the Google Cloud site. [ternary.app +5](#)

13.2 Services Overview

• Compute services

These services execute code, manage containers, or provide virtual servers (instances) to run applications. 

- **Amazon EC2 (Elastic Compute Cloud):** Provides resizable virtual servers (instances) for full control over the OS.
- [AWS Fargate](#): Serverless compute engine for containers that works with ECS/EKS.
- **Amazon EKS/ECS:** Managed Kubernetes or Docker container orchestration.
- [Auto Scaling](#): Automatically adjusts capacity based on demand.  LinkedIn

• Storage services

These services provide scalable, durable storage for objects, files, and blocks. 

- **Amazon S3 (Simple Storage Service):** Object storage optimized for high durability, performance, and scalability.
- **Amazon EBS (Elastic Block Store):** Persistent block storage volumes for EC2 instances.
- **Amazon EFS (Elastic File System):** Managed shared file system.
- **Amazon S3 Glacier:** Low-cost, long-term archival storage.  GeeksforGeeks +2

• Identity services

These services manage user access, authentication, and security permissions. 

- **AWS IAM (Identity and Access Management):** Manages users, permissions, and security credentials.
- **Amazon Cognito:** Adds user sign-up, sign-in, and access control to mobile/web apps.
- **AWS Directory Service:** Connects AWS resources to existing Microsoft Active Directory.  Scaler +1

- **Serverless services**

These services allow developers to build and run applications without managing infrastructure, with automatic scaling and pay-as-you-go pricing.  Microsoft Azure +1

- **AWS Lambda:** Runs code in response to events without provisioning servers.
- **Amazon DynamoDB:** Fully managed, serverless NoSQL database.
- **AWS Step Functions:** Orchestrates serverless workflows.
- ****Amazon EventBridge**

UNIT 14: Case Studies and Applications

14.1 Applications

- E-commerce systems on cloud
- Big Data applications
- IoT and Cloud integration